

LECTURE 10

CEN 348-Artificial Intelligence Systems
Solving problems by searching

Ezgi Zorarpacı

Problem solving agents

- Intelligent agents are supposed to maximize their performance measure
- This can be simplified if the agent can adopt a **goal** and aim at satisfying it
- Goals help organize behavior by limiting the objectives that the agent is trying to achieve
- **Goal formulation**, based on the current situation and the agent's performance measure, is the first step in problem solving
- Goal is a set of states. The agent's task is to find out which sequence of actions will get it to a goal state
- **Problem formulation** is the process of deciding what sorts of actions and states to consider, given a goal

Problem solving agents

- An agent with several immediate options of unknown value can decide what to do by first examining different possible sequences of actions that lead to states of known value, and then choosing the best sequence
- Looking for such a sequence is called **search**
- A search algorithm takes a problem as input and returns a **solution** in the form of action sequence
- Once a solution is found the actions it recommends can be carried out – **execution** phase
- “**formulate, search, execute**” design for the agent
- After formulating a goal and a problem to solve the agent calls a search procedure to solve it

Problem solving agents

- It then uses the solution to guide its actions, doing whatever the solution recommends as the next thing to do (typically the first action in the sequence)
- Then removing that step from the sequence
- Once the solution has been executed, the agent will formulate a new goal

Problem solving agents

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  static: seq, an action sequence, initially empty
           state, some description of the current world state
           goal, a goal, initially null
           problem, a problem formulation

  state ← UPDATE-STATE(state, percept)
  if seq is empty then do
    goal ← FORMULATE-GOAL(state)
    problem ← FORMULATE-PROBLEM(state, goal)
    seq ← SEARCH(problem)
  action ← FIRST(seq)
  seq ← REST(seq)
  return action
```

Environment Assumptions

- Static, formulating and solving the problem is done without paying attention to any changes that might be occurring in the environment
- Initial state is known and the environment is observable
- Discrete, enumerate alternative courses of actions
- Deterministic, solutions to problems are single sequences of actions, so they cannot handle any unexpected events, and solutions are executed without paying attention to the percepts

Example: Romania

On holiday in Romania; currently in Arad.

Flight leaves tomorrow from Bucharest

Formulate goal:

- be in Bucharest

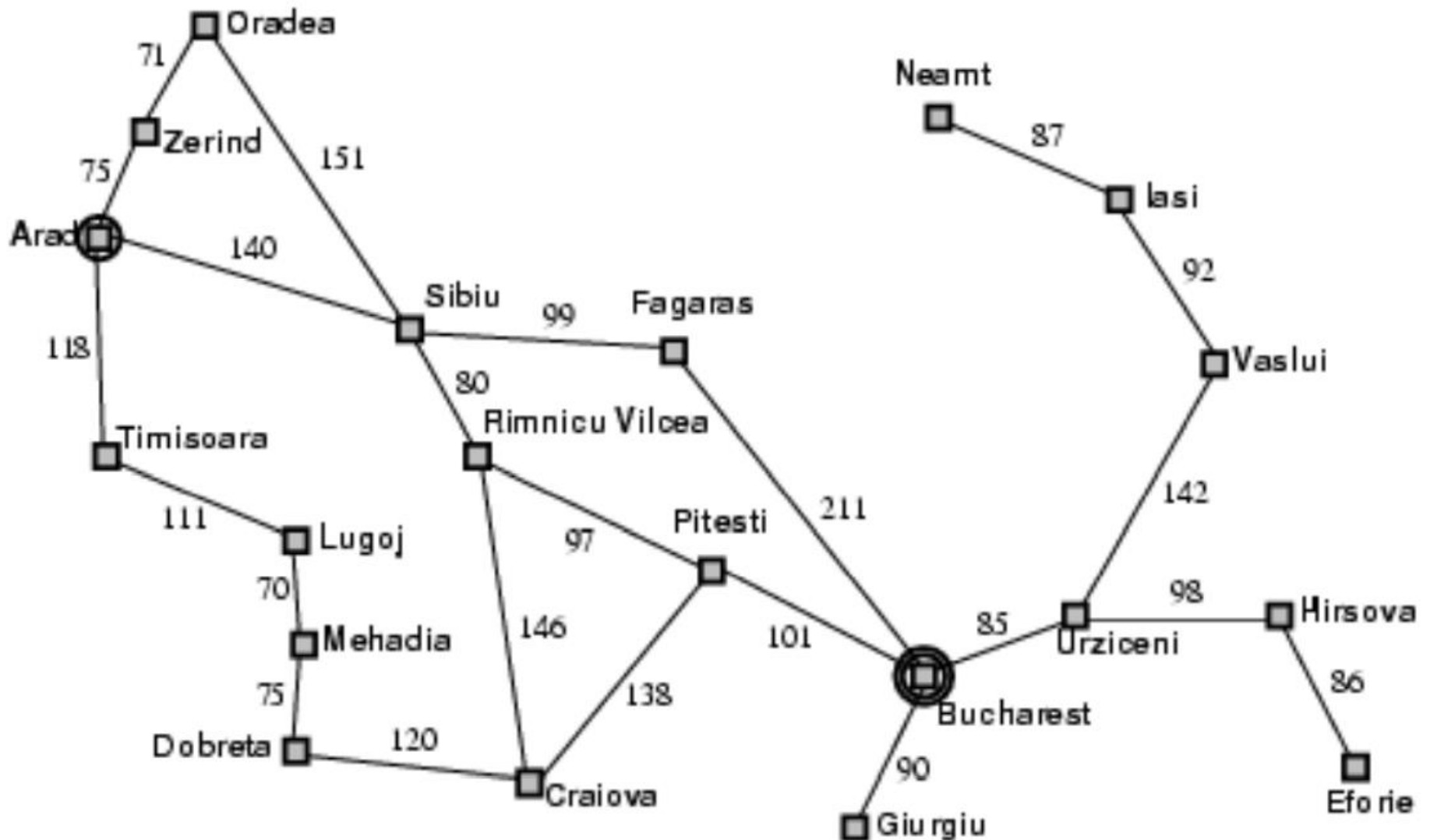
Formulate problem:

- **states**: various cities
- **actions**: drive between cities

Find solution:

- sequence of cities, e.g., Arad, Sibiu, Fagaras, Bucharest

Example: Romania



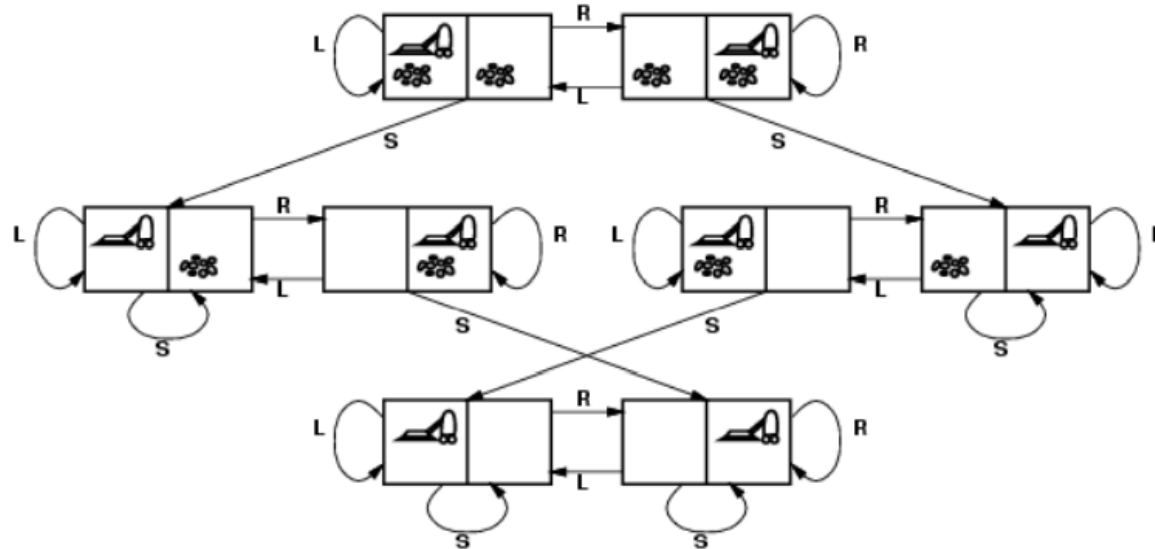
Example: Romania

- A problem can be defined formally by four components
- Initial state that the agent starts in
 - e.g. In(Arad)
- A description of the possible actions available to the agent
 - Successor function – returns a set of <action,successor> pairs
 - e.g. {<Go(Sibiu),In(Sibiu)>, <Go(Timisoara),In(Timisoara)>, <Go(Zerind), In(Zerind)>}
 - Initial state and the successor function define the state space (a graph in which the nodes are states and the arcs between nodes are actions). A path in state space is a sequence of states connected by a sequence of actions
- Goal test determines whether a given state is a goal state
 - e.g. {In(Bucharest)}
- Path cost function that assigns a numeric cost to each path. The cost of a path can be described as the some of the costs of the individual actions along the path – step cost
 - e.g. Time to go Bucharest

Problem Formulation

- A solution to a problem is a path from the initial state to the goal state
- Solution quality is measured by the path cost function and an optimal solution has the lowest path cost among all solutions
- Real world is absurdly complex
 - state space must be **abstracted** for problem solving
- (Abstract) state = set of real states
- (Abstract) action = complex combination of real actions
 - e.g., "Arad $\rightarrow$ Zerind" represents a complex set of possible routes, detours, rest stops, etc.
- For guaranteed realizability, **any** real state "in Arad" must get to **some** real state "in Zerind"
- (Abstract) solution =
 - set of real paths that are solutions in the real world
- Each abstract action should be "easier" than the original problem

Vacuum world state space graph



- states integer dirt and robot location.
 - The agent is in one of two locations, each of which might or might not contain dirt – 8 possible states
- Initial state: any state
- actions *Left, Right, Suck*
- goal test no dirt at all locations
- path cost 1 per action

Example: The 8-puzzle

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- states: locations of tiles
- Initial state: any state
- actions: move blank left, right, up, down
- goal test: goal state (given)
- path cost: 1 per move

[Note: optimal solution of n -Puzzle family is NP-hard]

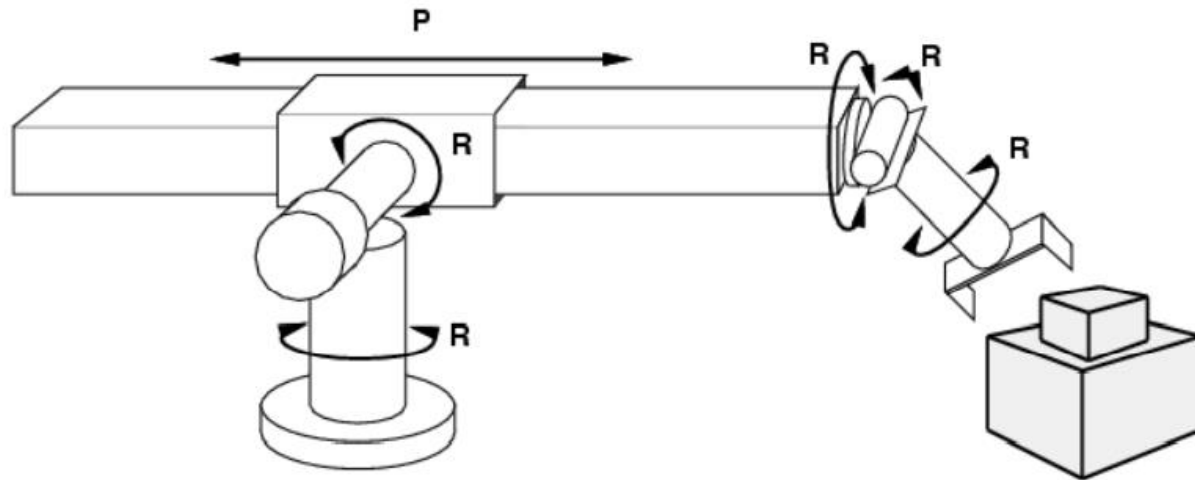
Example: Route finding problem

- states: each is represented by a location (e.g. An airport) and the current time
- Initial state: specified by the problem
- Successor function: returns the states resulting from taking any scheduled flight, leaving later than the current time plus the within airport transit time, from the current airport to another
- goal test: are we at the destination by some pre-specified time
- Path cost: monetary cost, waiting time, flight time, customs and immigration procedures, seat quality, time of day, type of airplane, frequent-flyer mileage awards, etc
- Route finding algorithms are used in a variety of applications, such as routing in computer networks, military operations planning, airline travel planning systems

Other example problems

- Touring problems: visit every city at least once, starting and ending at Bucharest
- Travelling salesperson problem (TSP) : each city must be visited exactly once – find the shortest tour
- VLSI layout design: positioning millions of components and connections on a chip to minimize area, minimize circuit delays, minimize stray capacitances, and maximize manufacturing yield
- Robot navigation
- Internet searching
- Automatic assembly sequencing
- Protein design

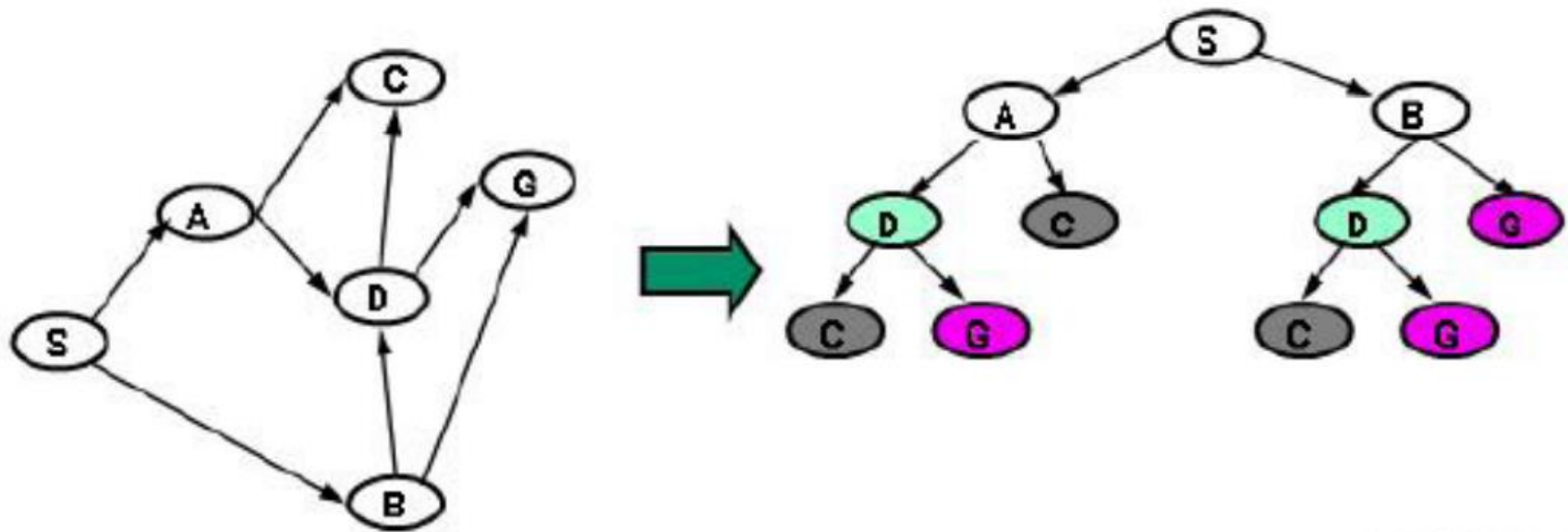
Example: robotic assembly



- states: real-valued coordinates of robot joint angles parts of the object to be assembled
- actions: continuous motions of robot joints
- goal test: complete assembly
- path cost: time to execute

Graph Search as Tree Search

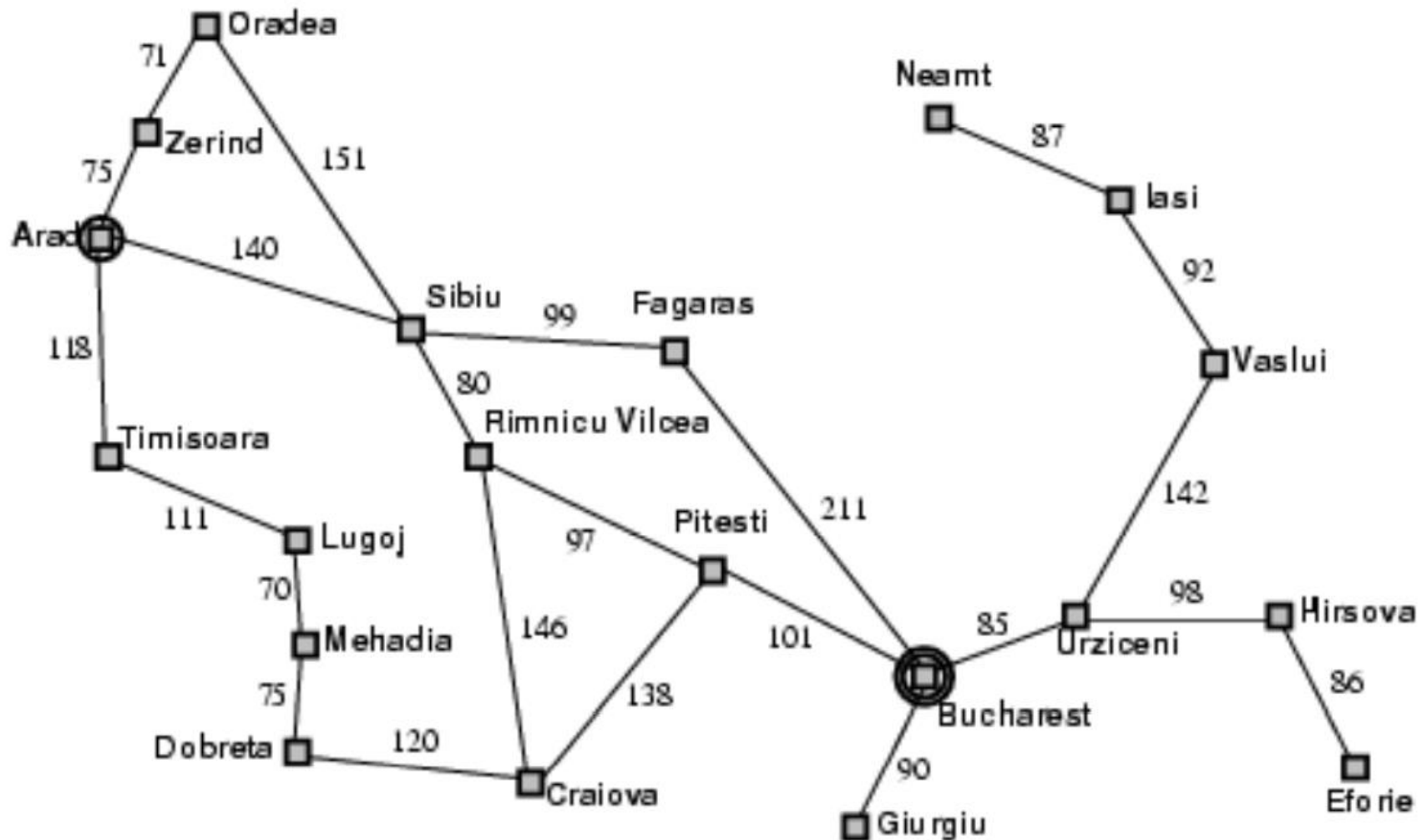
- Trees are directed graphs without cycles and with nodes having ≤ 1 parent
- We can turn graph search problems (from S to G) into tree search problems by:
 - replacing undirected links by 2 directed links
 - avoiding loops in path (or keeping track of visited nodes globally)



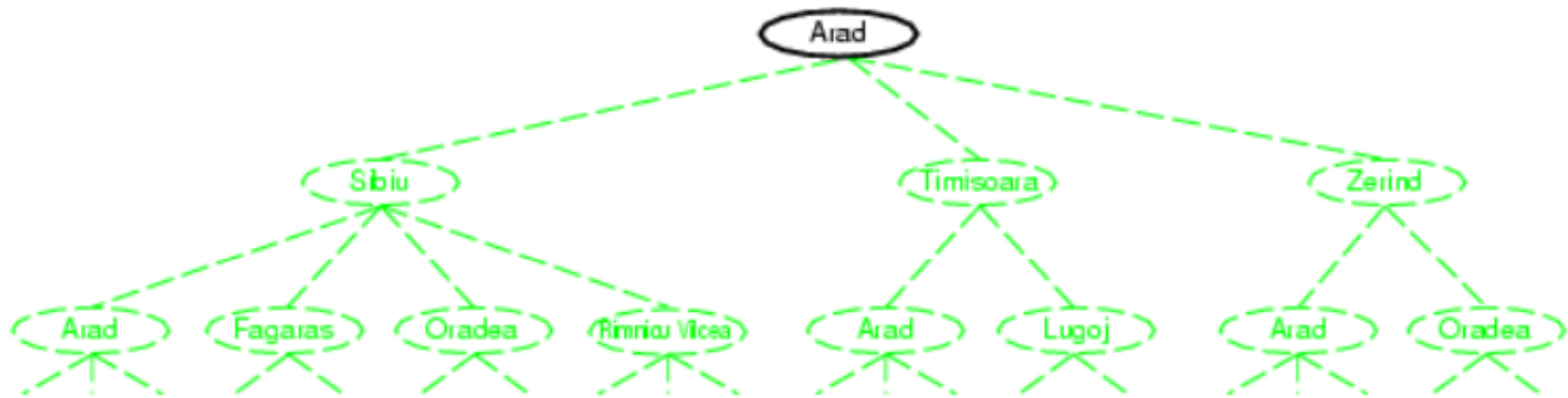
Terminology

- **State** – Used to refer to the vertices of the underlying **graph** that is being searched, that is, states in the problem domain, for example, a city, an arrangement of blocks or the arrangement of parts in a puzzle.
- **Search Node** – Refers to the vertices of the search **tree** which is being generated by the search algorithm. Each node refers to a state of the world; **many nodes may refer to the same state**. Importantly, a node implicitly represents a path (from the start state of the search to the state associated with the node). Because search nodes are part of a search tree, they have a unique ancestor node (except for the root node).

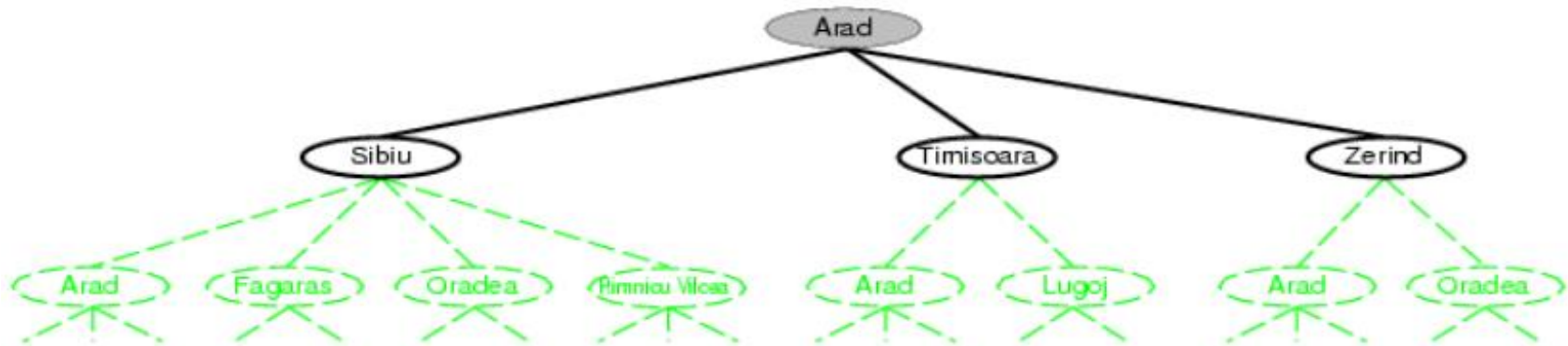
Example: Romania



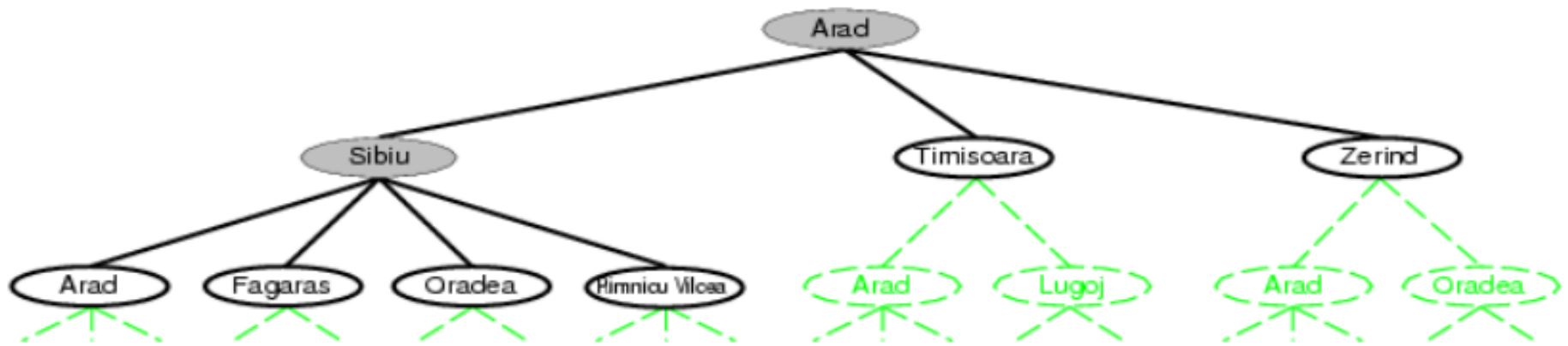
Tree search example



Tree search example



Tree search example

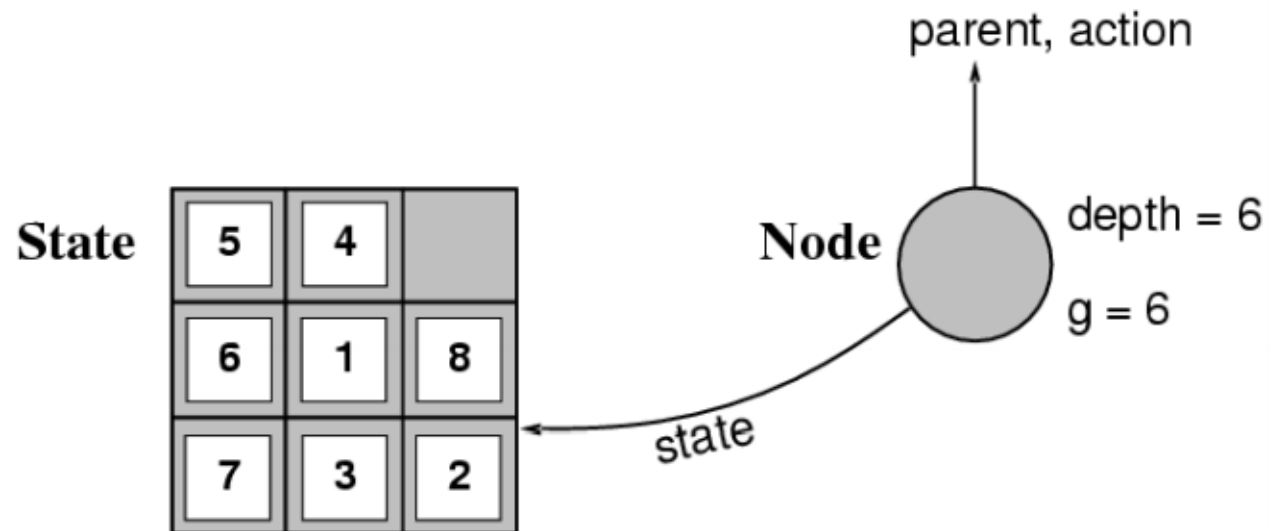


Implementation: Components of a node

- **State:** the state in the state space to which the node corresponds
- **Parent-node:** the node in the search tree that generated this node
- **Action:** the action that was applied to the parent to generate the node
- **Path-cost:** the cost, traditionally denoted by $g(n)$, of the path from the initial state to the node, as indicated by the parent pointers
- **Depth:** the number of steps along the path from the initial state

Implementation: states vs. nodes

- A **state** is a (representation of) a physical configuration
- A **node** is a data structure constituting part of a search tree includes **state**, **parent node**, **action**, **path cost** $g(x)$, **depth**



Implementation: general tree search

Fringe: the collection of nodes that have been generated but not yet been expanded

Each element of a fringe is a leaf node, a node with no successors

Search strategy: a function that selects the next node to be expanded from fringe

We assume that the collection of nodes is implemented as a queue

The operations on the queue are:

- Make-queue(queue)
- Empty?(queue)
- first(queue)
- remove-first(queue)
- insert(element, queue)
- insert-all(elements, queue)

Implementation: general tree search

```
function TREE-SEARCH(problem, fringe) returns a solution, or failure  
  fringe  $\leftarrow$  INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)  
  loop do  
    if fringe is empty then return failure  
    node  $\leftarrow$  REMOVE-FRONT(fringe)  
    if GOAL-TEST[problem](STATE[node]) then return SOLUTION(node)  
    fringe  $\leftarrow$  INSERTALL(EXPAND(node, problem), fringe)
```

```
function EXPAND(node, problem) returns a set of nodes  
  successors  $\leftarrow$  the empty set  
  for each action, result in SUCCESSOR-FN[problem](STATE[node]) do  
    s  $\leftarrow$  a new NODE  
    PARENT-NODE[s]  $\leftarrow$  node; ACTION[s]  $\leftarrow$  action; STATE[s]  $\leftarrow$  result  
    PATH-COST[s]  $\leftarrow$  PATH-COST[node] + STEP-COST(node, action, s)  
    DEPTH[s]  $\leftarrow$  DEPTH[node] + 1  
    add s to successors  
  return successors
```

- The Expand function creates new nodes, filling in the various fields and using the SuccessorFn of the problem to create the corresponding states.

Examples: Simple search strategies

- Depth first search
 - Pick first element of Q
 - Add path extensions to front of Q
- Breadth first search
 - Pick first element of Q
 - Add path extensions to the end of Q

Visited versus expanded

- **Visited:** a state M is first visited when a path to M first gets added to Q . In general, a state is said to have been visited if it has ever shown up in a search node in Q . The intuition is that we have briefly visited them to place them on Q , but we have not yet examined them carefully
- **Expanded:** a state M is expanded when it is the state of a search node that is pulled off of Q . At that point, the descendants of M are visited and the path that led to M is extended to the eligible descendants. In principle, a state may be expanded multiple times. We sometimes refer to the search node that led to M as being expanded. However, once a node is expanded, we are done with it, we will not need to expand it again. In fact, we discard it from Q

Testing for the goal

- This algorithm stops (in step 3) when $\text{state}(N) = G$ or, in general when $\text{state}(N)$ satisfies the goal test
- We could have performed the test in step 6 as each extended path is added to Q . This would catch termination earlier
- However, performing the test in step 6 will be incorrect for the optimal searches

Keeping track of visited states

Keeping track of visited states generally improves time efficiency when searching for graphs, without affecting correctness. Note, however, that substantial additional space may be required to keep track of visited states.

If all we want to do is find a path from the start to goal, there is no advantage of adding a search node whose state is already the state of another search node

Any state reachable from the node the second time would have been reachable from that node the first time

Note that when using Visited, each state will only ever have at most one path to it (search node) in Q

We'll have to revisit this issue when we look at optimal searching

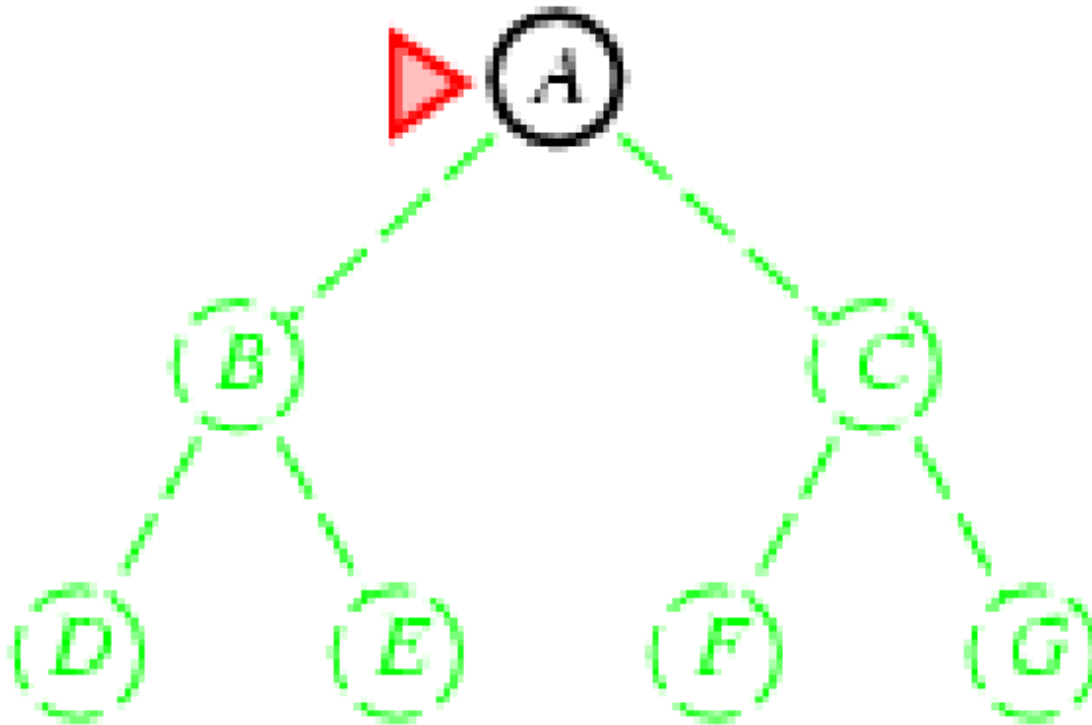
Implementation issues : The visit List

- Although we speak of a visited list, this is never the preferred implementation
- If the graph states are known ahead of time as an explicit set, then space is allocated in the state itself to keep a mark, which makes both adding Visited and checking if a state is Visited a constant time operation
- Alternatively, as in more common AI, if the states are generated on the fly, then a hash table may be used for efficient detection of previously visited states.
- Note that, in any case, the incremental space cost of a Visited list will be proportional to the number of states, which can be very high in some problems

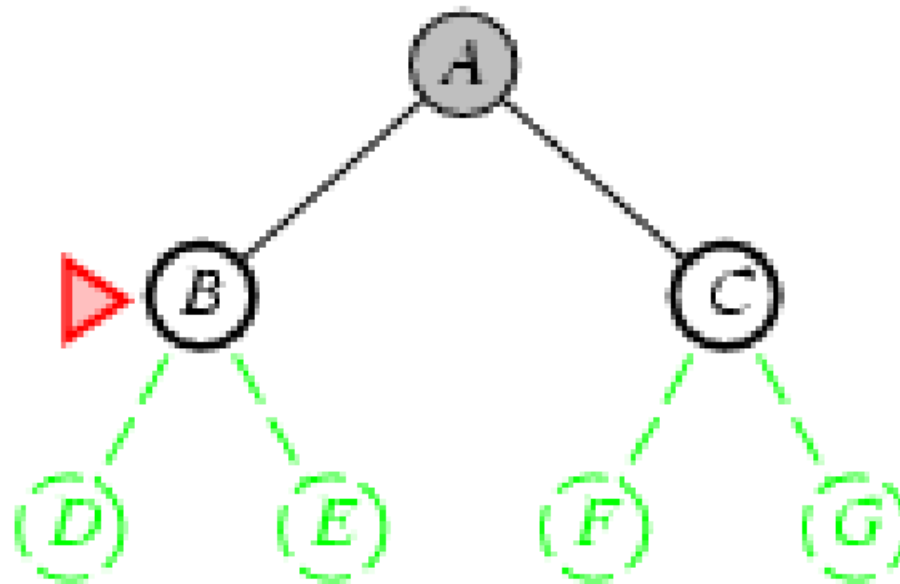
Breadth-first search

- The root node is expanded first, then all the successors of the root node, and their successors and so on
- In general, all the nodes are expanded at a given depth in the search tree before any nodes at the next level are expanded
- Expand shallowest unexpanded node
- **Implementation:**
 - *fringe* is a FIFO queue,
 - the nodes that are visited first will be expanded first
 - All newly generated successors will be put at the end of the queue
 - Shallow nodes are expanded before deeper nodes

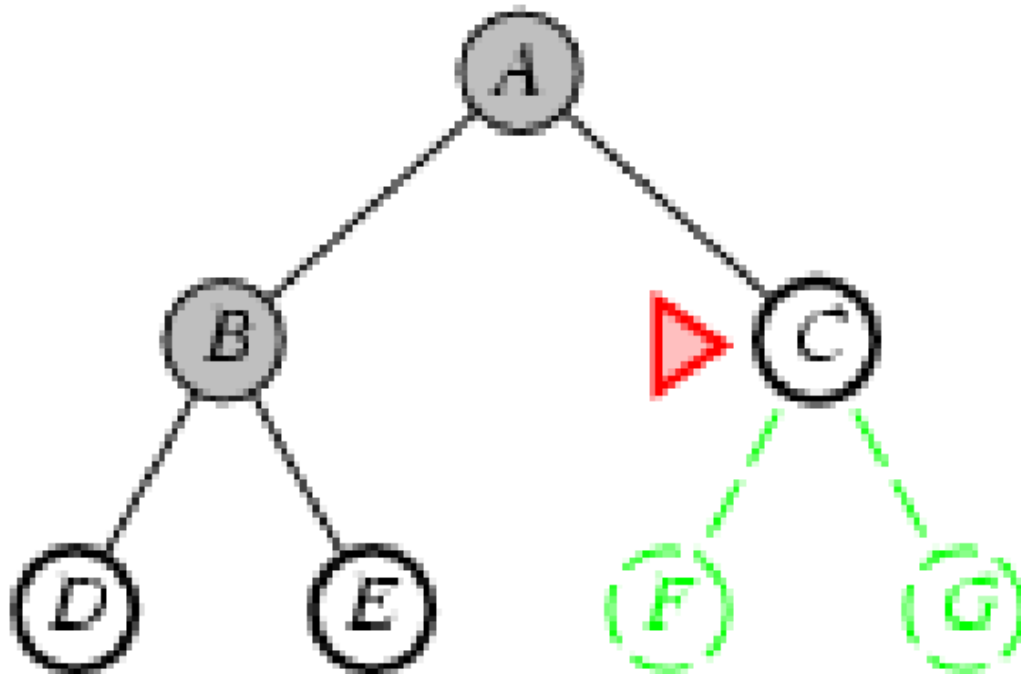
Breadth-first search



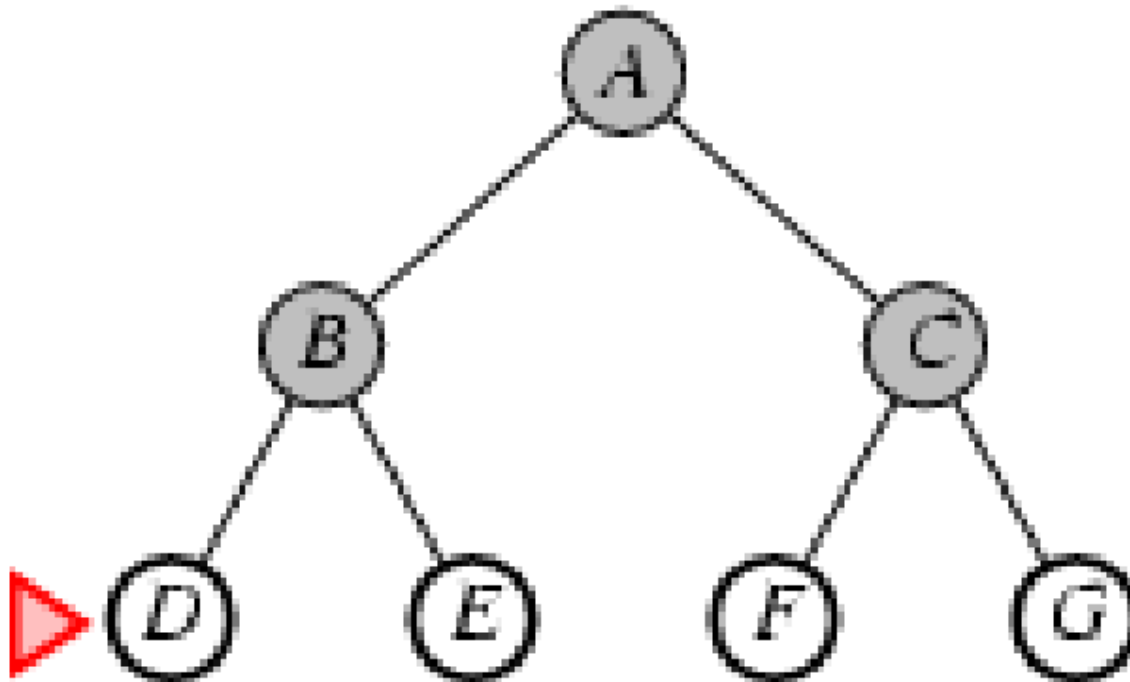
Breadth-first search



Breadth-first search



Breadth-first search



Example: 8-puzzle problem

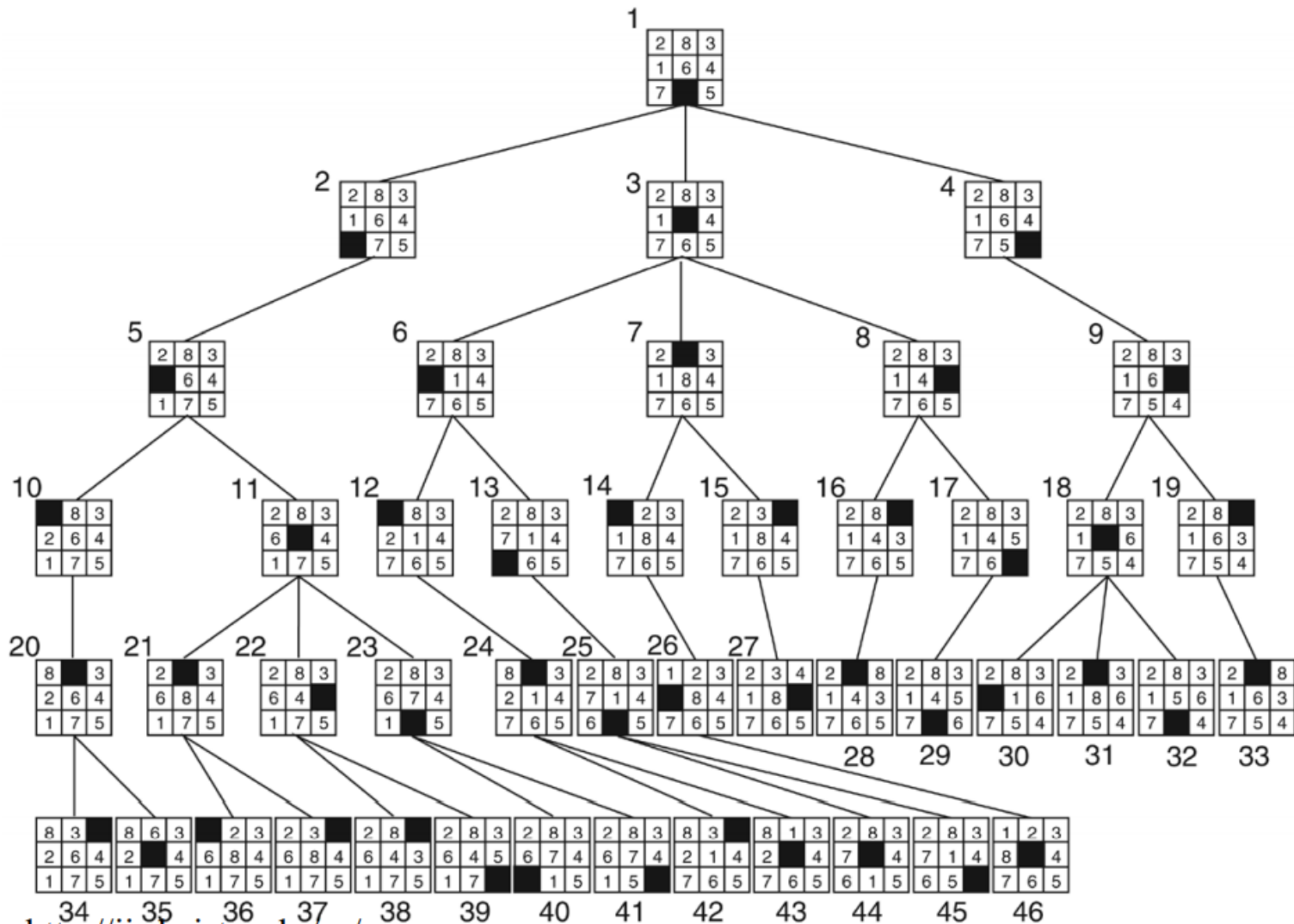
<https://www.youtube.com/watch?v=7O4izELjAw0>

2	8	3
1	6	4
7		5



1	2	3
8		4
7	6	5

Example: 8-puzzle problem with BFS



Taken from <http://iis.kaist.ac.kr/es/>

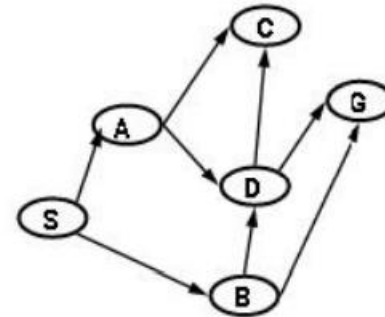
Goal

BFS and DFS videos

- <https://www.youtube.com/watch?v=4Fv5cQfsRDg>
- <https://www.youtube.com/watch?v=hmEu4Z4XRkl&t=0s>

Breadth first search

	Q	Visited
1	(S)	S
2		
3		
4		
5		
6		



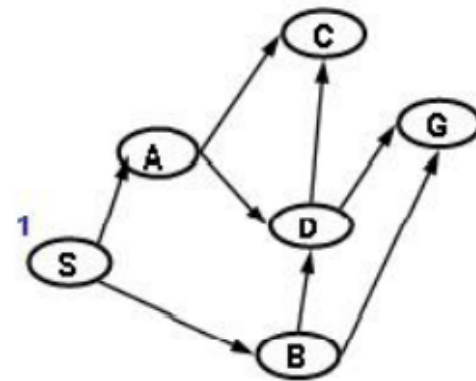
Pick first element of Q; Add path extensions to end of Q

Added paths in **blue**

We show the paths in **reversed** order; the node's state is the first entry.

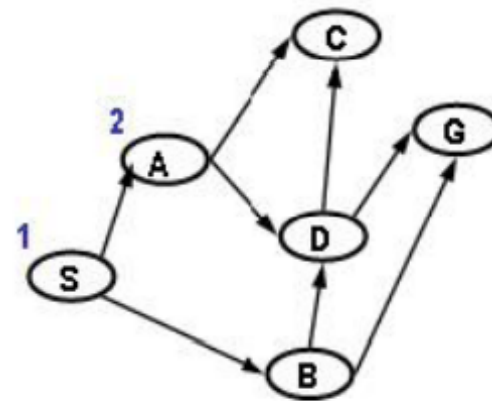
Breadth first search

	Q	Visited
1	(S)	S
2	(A S) (B S)	A,B,S
3		
4		
5		
6		



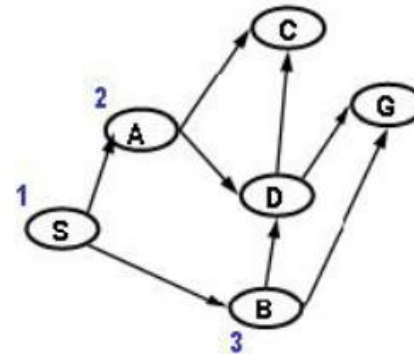
Breadth first search

	Q	Visited
1	(S)	S
2	(A S) (B S)	A,B,S
3	(B S) (C A S) (D A S)	C,D,B,A,S
4		
5		
6		



Breadth first search

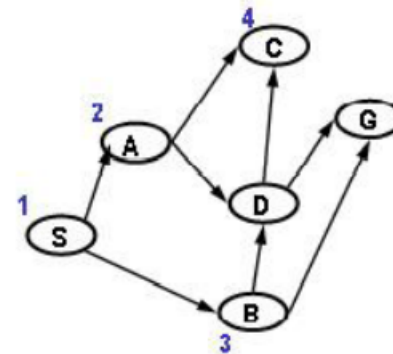
	Q	Visited
1	(S)	S
2	(A S) (E S)	A,B,S
3	(B S) (C A S) (D A S)	C,D,B,A,S
4	(C A S) (D A S) (G B S)*	G,C,D,B,A,S
5		
6		



* We could have stopped here, when the first path to the goal was generated.

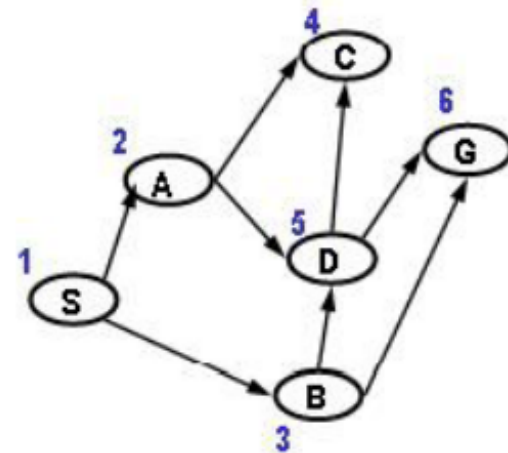
Breadth first search

	Q	Visited
1	(S)	S
2	(A S) (B S)	A,B,S
3	(B S) (C A S) (D A S)	C,D,B,A,S
4	(C A S) (D A S) (G B S)*	G,C,D,B,A,S
5	(D A S) (G B S)	G,C,D,B,A,S
6		



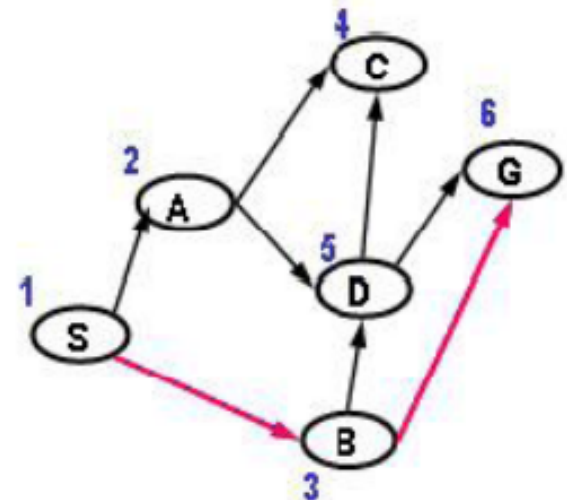
Breadth first search

	Q	Visited
1	(S)	S
2	(A S) (B S)	A,B,S
3	(B S) (C A S) (D A S)	C,D,B,A,S
4	(C A S) (D A S) (G B S)*	G,C,D,B,A,S
5	(D A S) (G B S)	G,C,D,B,A,S
6	(G B S)	G,C,D,B,A,S

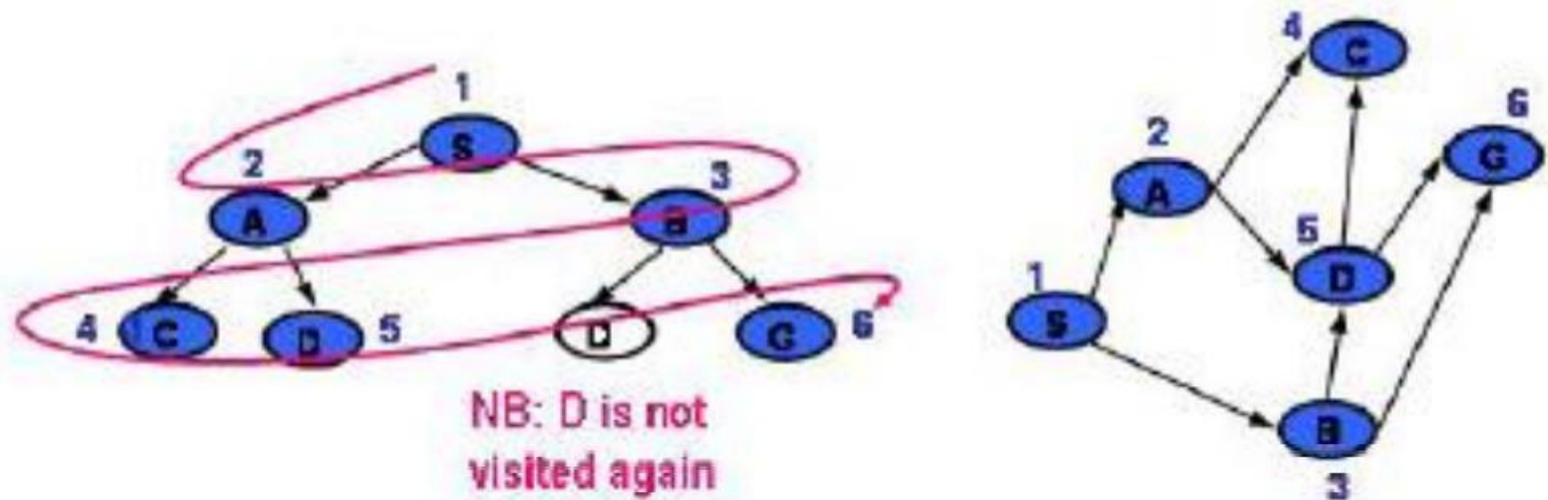


Breadth first search

	Q	Visited
1	(S)	S
2	(A S) (B S)	A,B,S
3	(B S) (C A S) (D A S)	C,D,B,A,S
4	(C A S) (D A S) (G B S)*	G,C,D,B,A,S
5	(D A S) (G B S)	G,C,D,B,A,S
6	(G B S)	G,C,D,B,A,S



Breadth first search



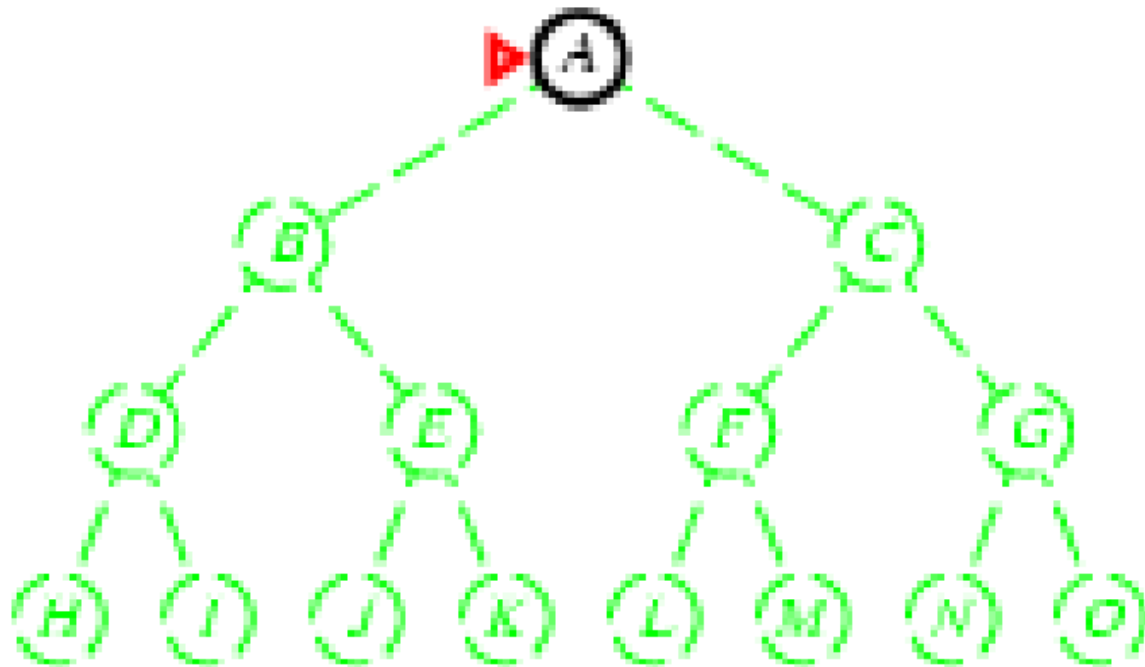
Numbers indicate order pulled off of Q (expanded)

Dark blue fill = Visited & Expanded

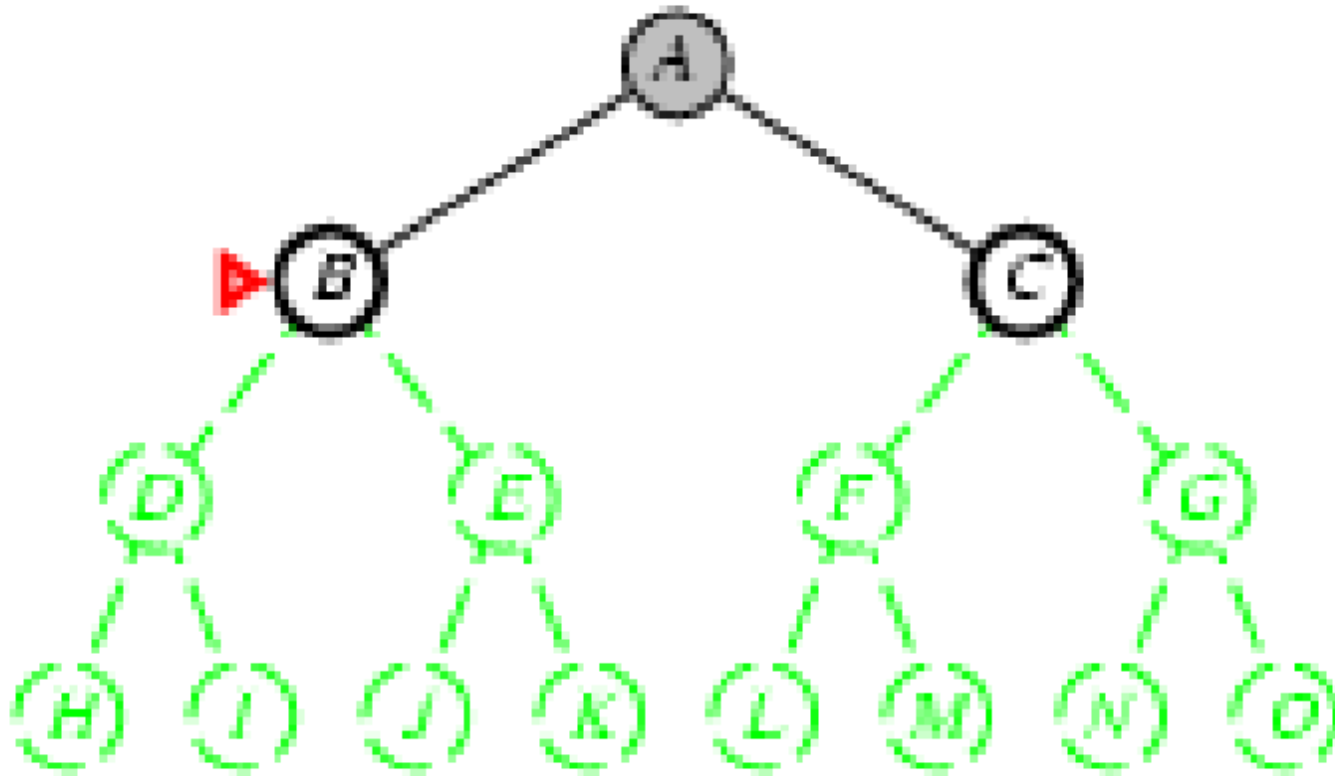
Light gray fill = Visited

Depth-first search

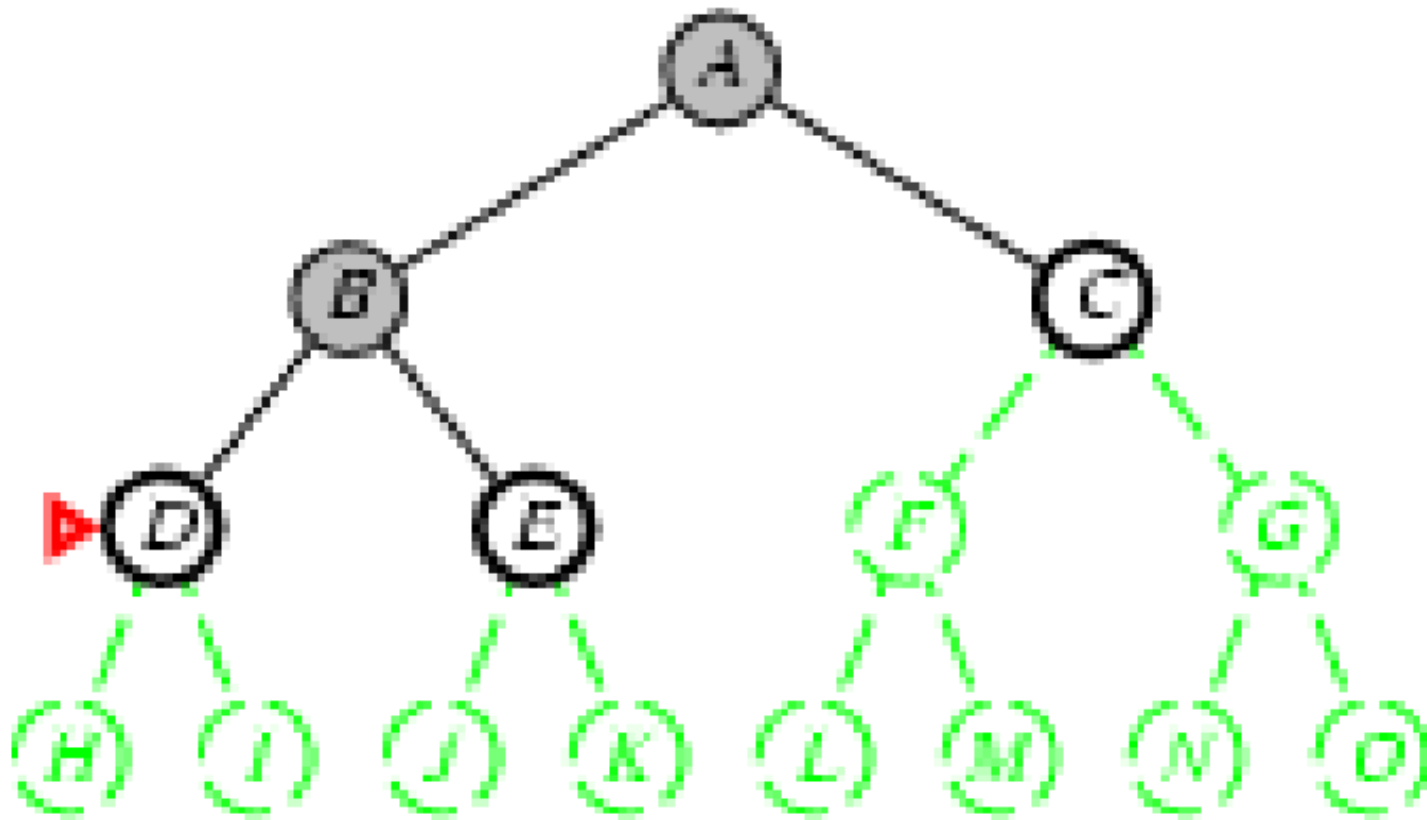
- Expand deepest unexpanded node
- **Implementation:**
 - *fringe* = LIFO queue (stack) , i.e., put successors at front



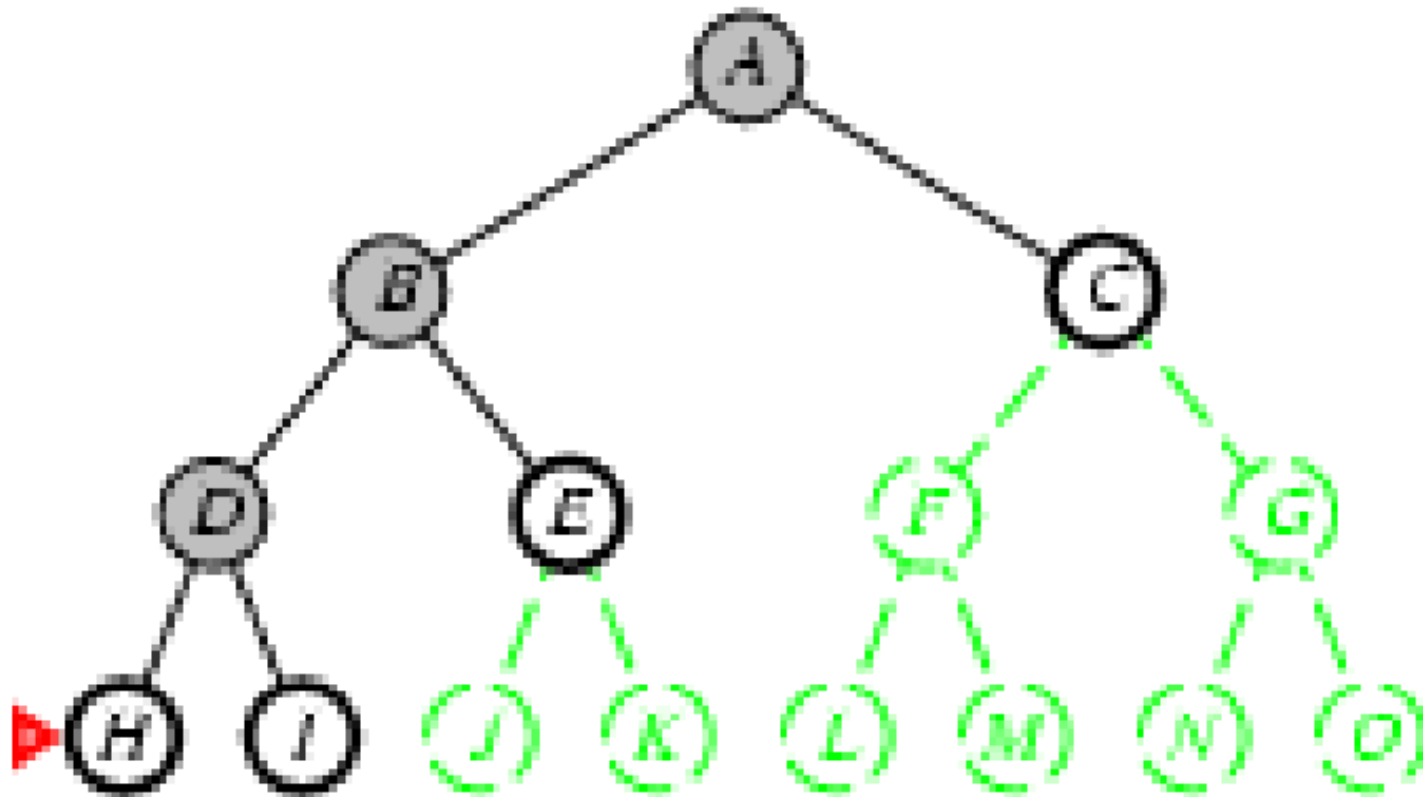
Depth-first search



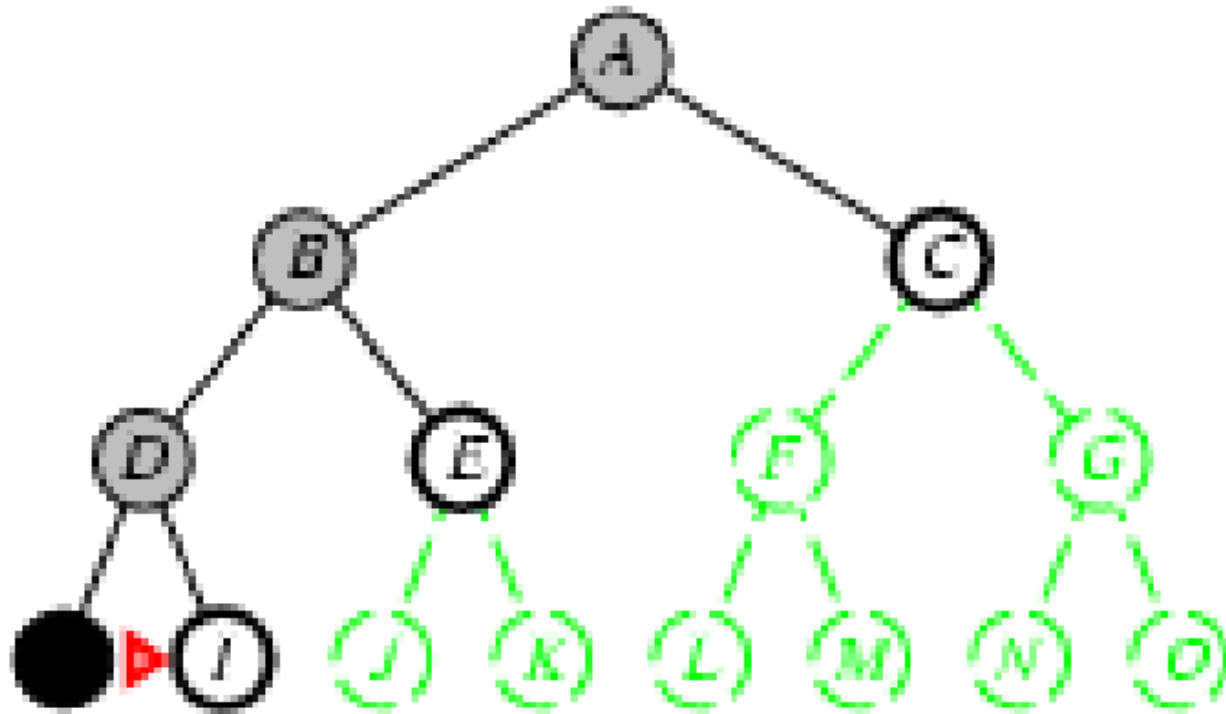
Depth-first search



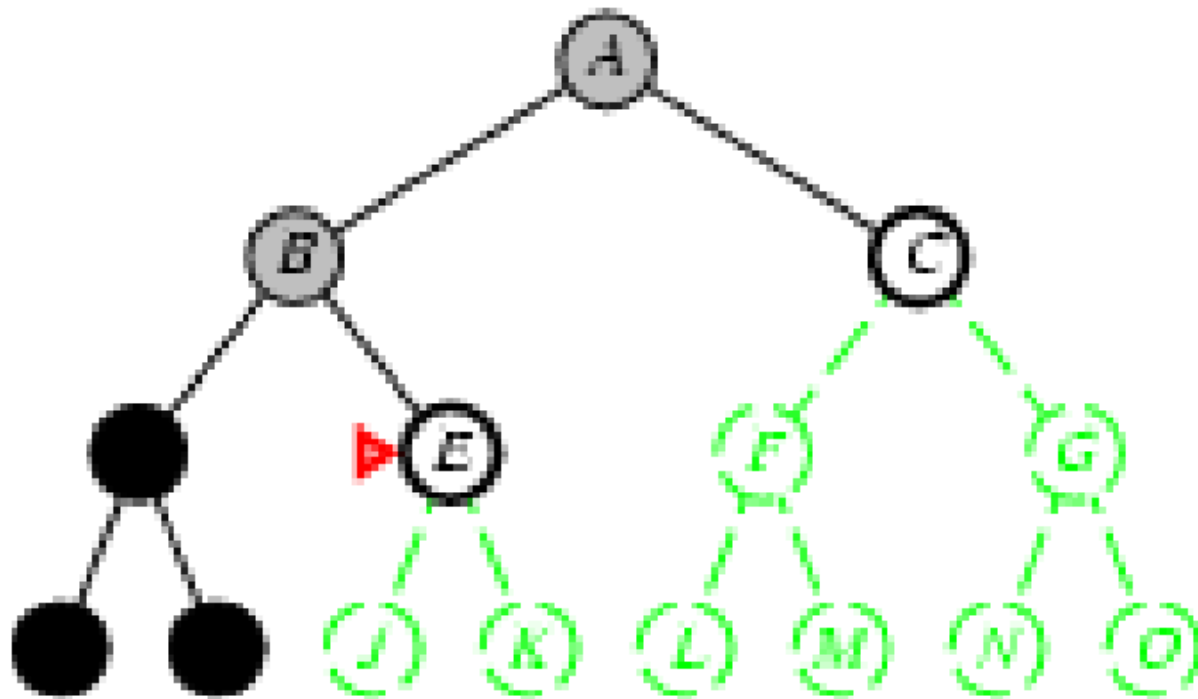
Depth-first search



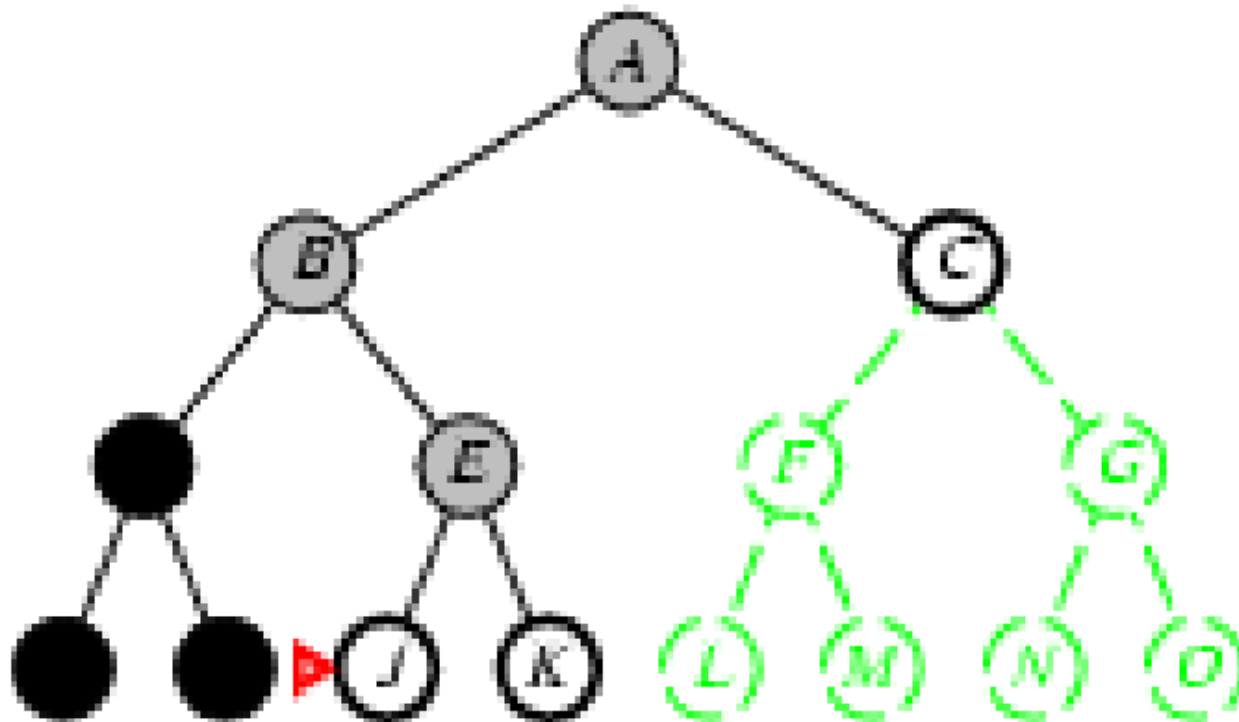
Depth-first search



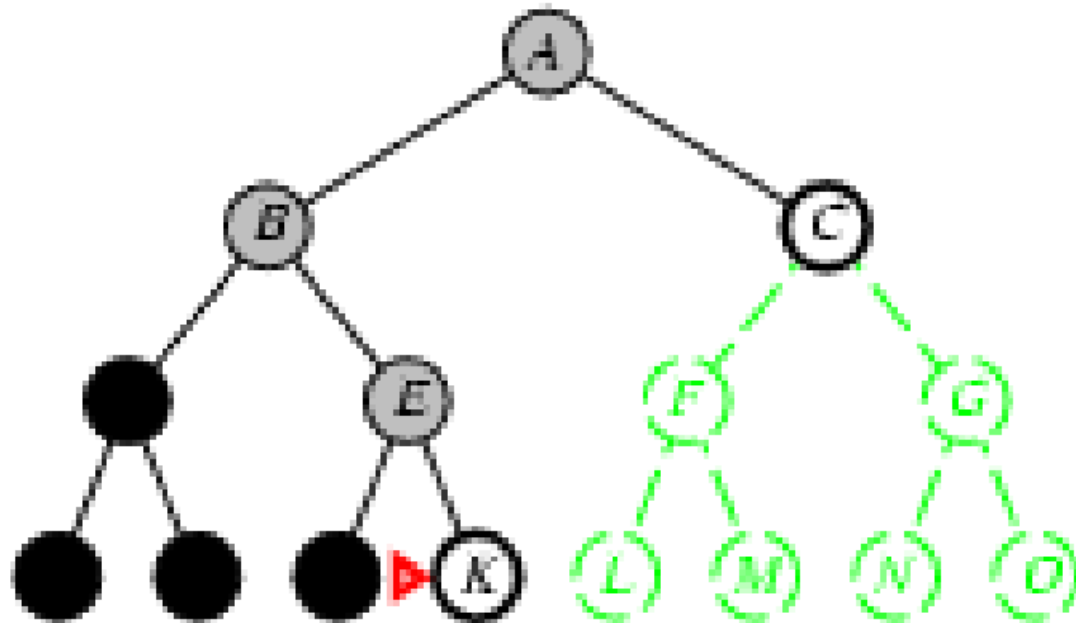
Depth-first search

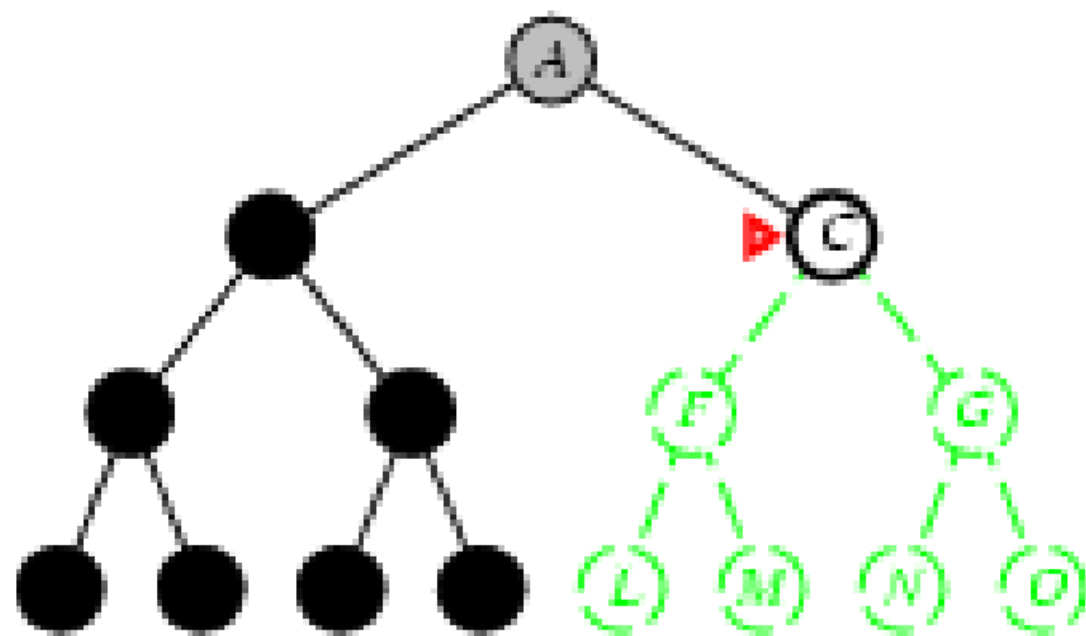


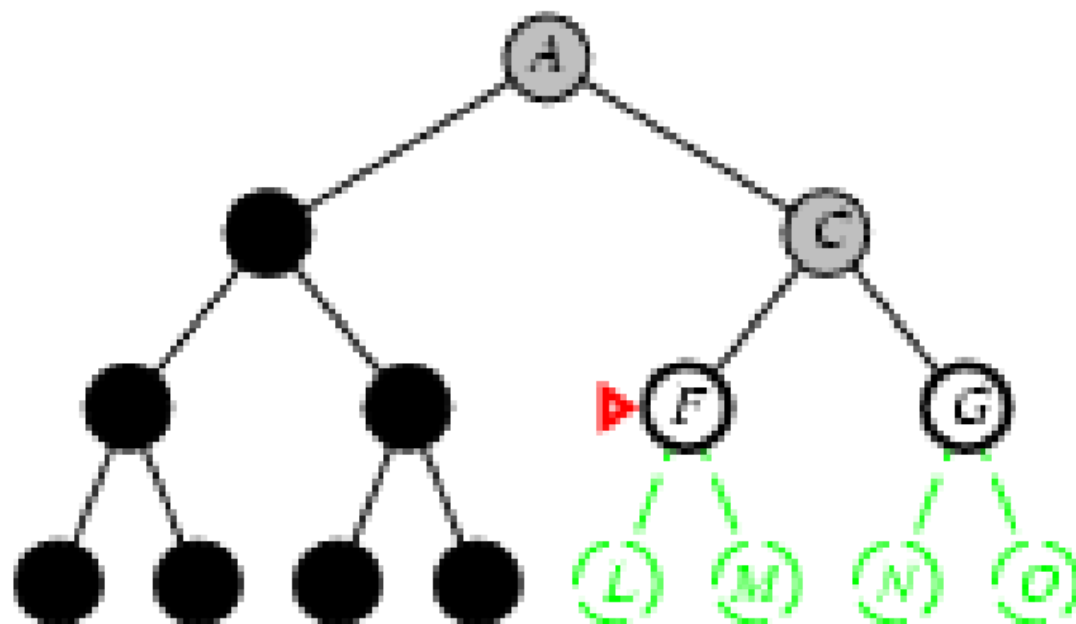
Depth-first search

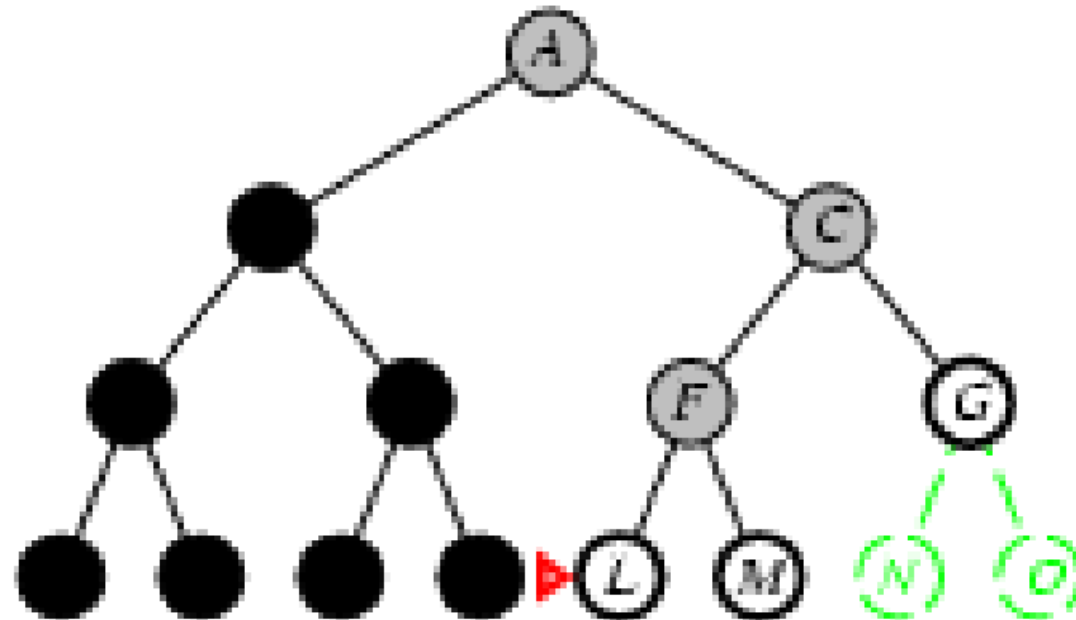


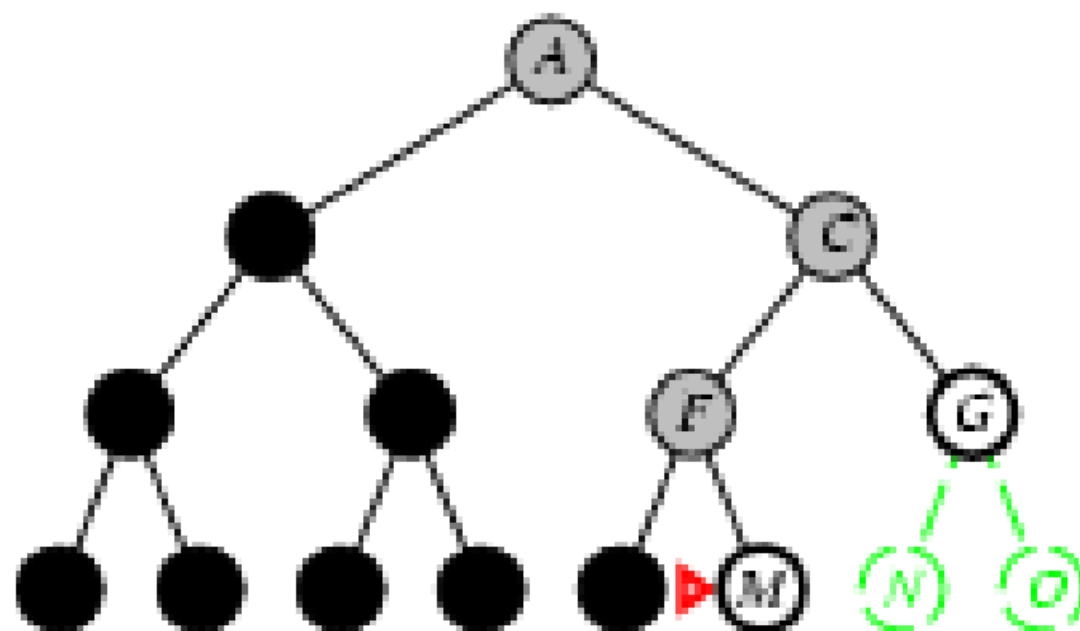
Depth-first search





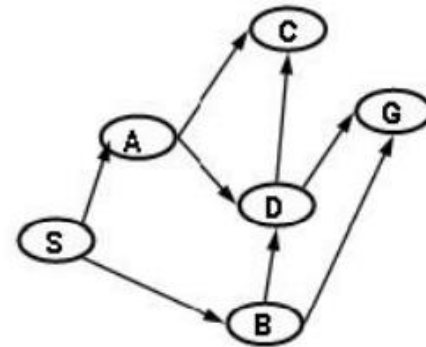






Depth first search

	Q	Visited
1		
2		
3		
4		
5		



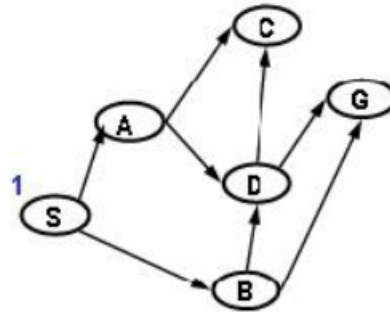
Pick first element of Q; Add path extensions to front of Q

Added paths in **blue**

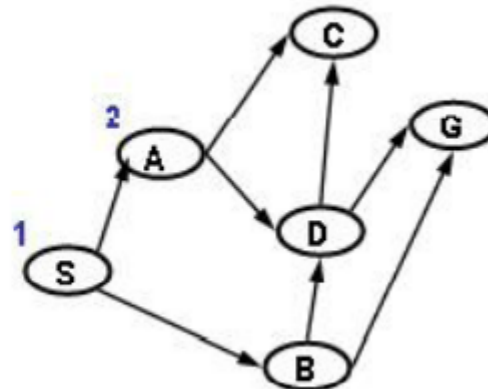
We show the paths in **reversed** order; the node's state is the first entry.

Depth first search

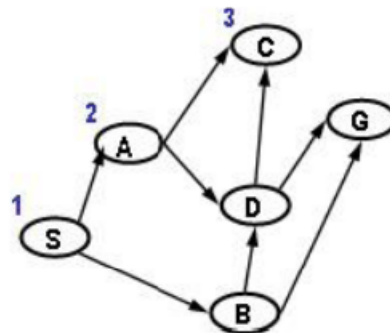
	Q	Visited
1	(S)	S
2		
3		
4		
5		



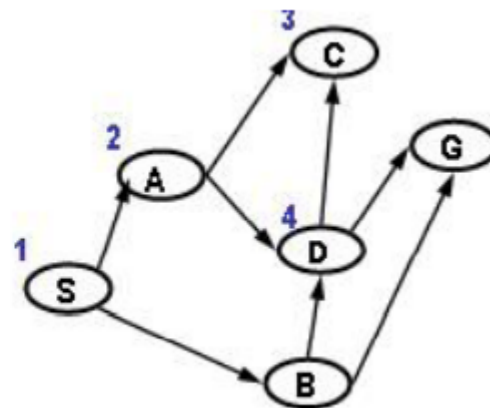
	Q	Visited
1	(S)	S
2	(A S) (B S)	A, B, S
3		
4		
5		



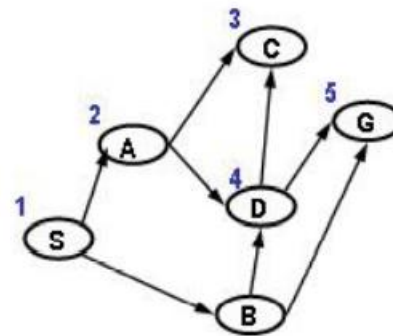
	Q	Visited
1	(S)	S
2	(A S) (B S)	A, B, S
3	(C A S) (D A S) (B S)	C, D, B, A, S
4		
5		



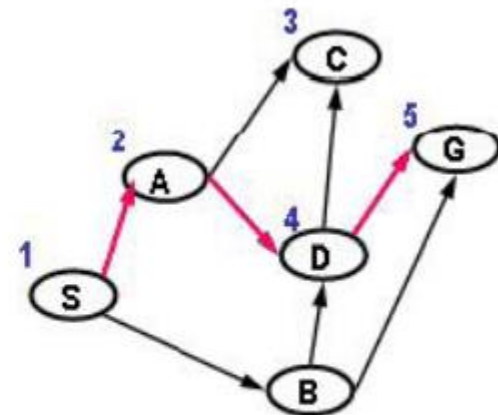
	Q	Visited
1	(S)	S
2	(A S) (B S)	A, B, S
3	(C A S) (D A S) (B S)	C, D, B, A, S
4	(D A S) (B S)	C, D, B, A, S
5		



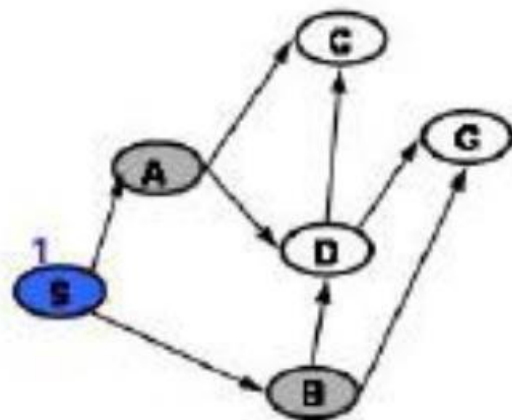
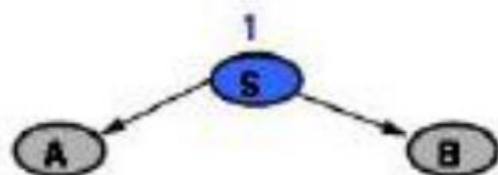
	Q	Visited
1	(S)	S
2	(A S) (B S)	A, B, S
3	(C A S) (D A S) (B S)	C, D, B, A, S
4	(D A S) (B S)	C, D, B, A, S
5	(G D A S) (B S)	G, C, D, B, A, S



	Q	Visited
1	(S)	S
2	(A S) (B S)	A, B, S
3	(C A S) (D A S) (B S)	C, D, B, A, S
4	(D A S) (B S)	C, D, B, A, S
5	(G D A S) (B S)	G, C, D, B, A, S



Another (easier?) way to see it

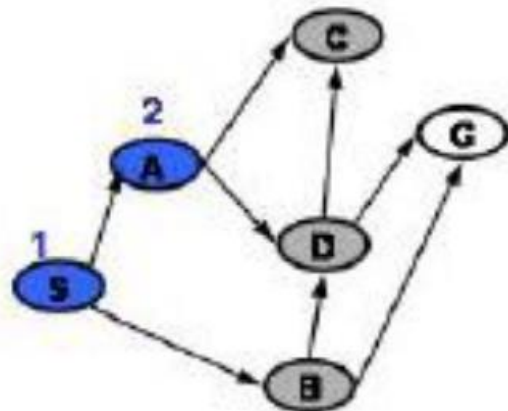
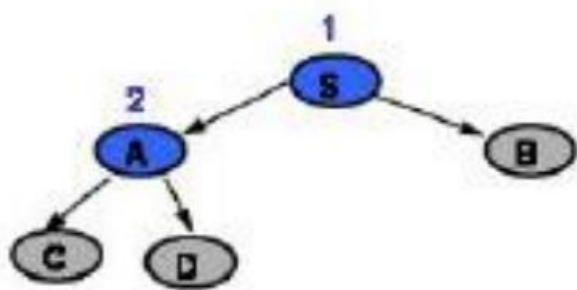


Numbers indicate order pulled off of Q (expanded)

Dark blue fill = Visited & Expanded

Light gray fill = Visited

Another (easier?) way to see it

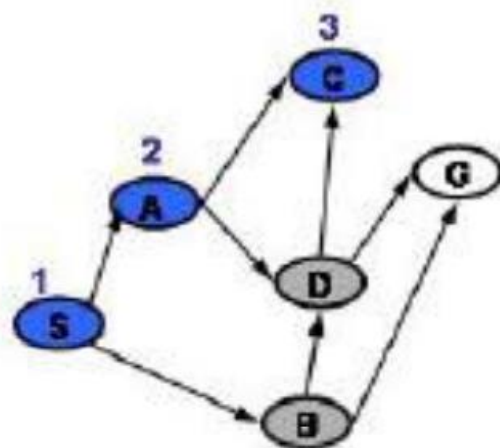
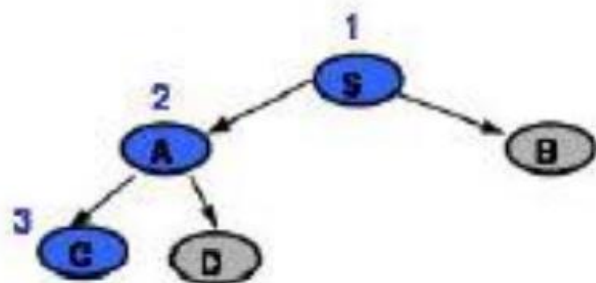


Numbers indicate order pulled off of Q (expanded)

Dark blue fill = Visited & Expanded

Light gray fill = Visited

Another (easier?) way to see it

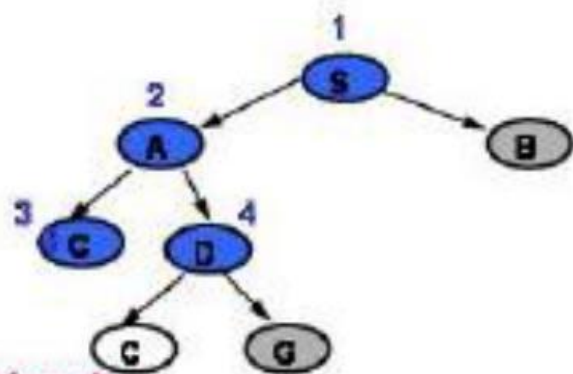


Numbers indicate order pulled off of Q (expanded)

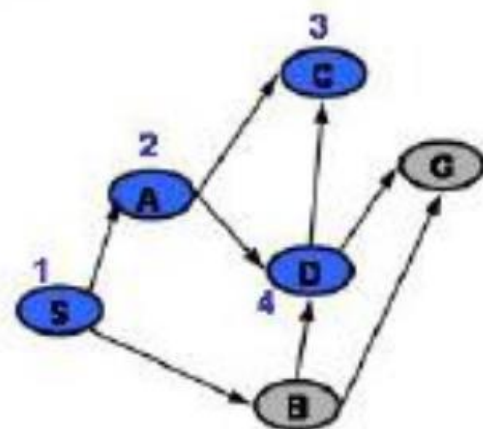
Dark blue fill = Visited & Expanded

Light gray fill = Visited

Another (easier?) way to see it



NB: C is not
visited again

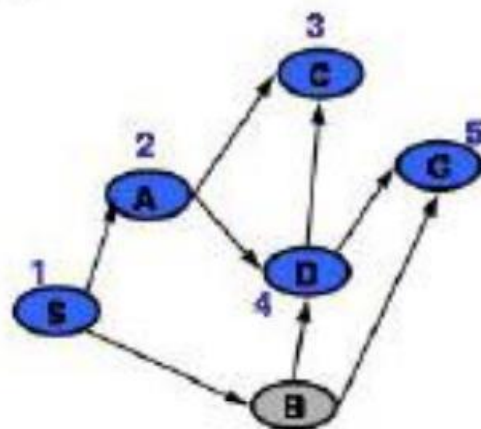
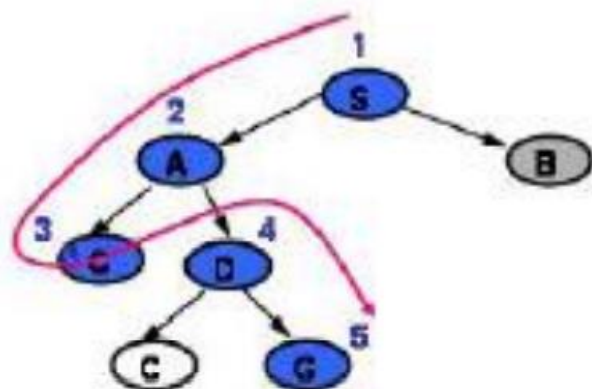


Numbers indicate order pulled off of Q (expanded)

Dark blue fill = Visited & Expanded

Light gray fill = Visited

Another (easier?) way to see it



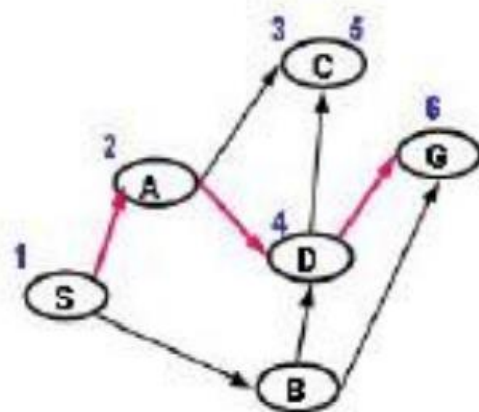
Numbers indicate order pulled off of Q (expanded)

Dark blue fill = Visited & Expanded

Light gray fill = Visited

Pick first element of Q; Add path extensions to front of Q

	Q
1	(S)
2	(A S) (B S)
3	(C A S) (D A S) (B S)
4	(D A S) (B S)
5	(C D A S) (G D A S) (B S)
6	(G D A S) (B S)



Added paths in blue

We show the paths in **reversed** order; the node's state is the first entry.

Do not extend a path to a state if the resulting path would have a loop.

Graph search

```
function GRAPH-SEARCH(problem, fringe) returns a solution, or failure
  closed ← an empty set
  fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node ← REMOVE-FRONT(fringe)
    if GOAL-TEST[problem](STATE[node]) then return SOLUTION(node)
    if STATE[node] is not in closed then
      add STATE[node] to closed
      fringe ← INSERTALL(EXPAND(node, problem), fringe)
```

Uninformed search strategies

- A **search strategy** is defined by picking the order of node expansion
- **Uninformed** search (blind search) strategies use only the information available in the problem definition
- **Can only distinguish goal states from non-goal state**
- Breadth-first search
- Uniform-cost search
- Depth-first search
- Depth-limited search
- Iterative deepening search

Analysis of Search strategies

- Strategies are evaluated along the following dimensions:
 - **completeness**: does it always find a solution if one exists?
 - **time complexity**: number of nodes generated
 - **space complexity**: maximum number of nodes in memory
 - **optimality**: does it always find a least-cost solution?
- Time and space complexity are measured in terms of
 - b : maximum branching factor of the search tree
 - d : depth of the least-cost solution
 - m : maximum depth of the state space (may be ∞)

Properties of breadth-first search

- Complete? Yes (if b is finite)
- Time?
Number of nodes in a b -ary tree of depth d :
(d is the depth of the optimal solution)
 $1 + b + b^2 + b^3 + \dots + b^d = O(b^d)$
- Space? $O(b^d)$ (keeps every node in memory)
- Optimal? Yes (if cost = 1 per step)
- **Space** is the bigger problem (more than time)

Depth	Nodes	Time	Memory
0	1	1 millisecond	100 kbytes
2	111	0.1 second	11 kilobytes
4	11,111	11 seconds	1 megabyte
6	10^6	18 minutes	111 megabytes
8	10^8	31 hours	11 gigabytes
10	10^{10}	128 days	1 terabyte
12	10^{12}	35 years	111 terabytes
14	10^{14}	3500 years	11,111 terabytes

Properties of depth-first search

- Complete? No: fails in infinite-depth spaces, spaces with loops
 - Modify to avoid repeated states along path
 - complete in finite spaces
- Time?

Could be the time to reach a solution at maximum depth m : $O(b^m)$

Terrible if m is much larger than d

But if there are lots of solutions, may be much faster than BFS
- Space? $O(bm)$, i.e., linear space!
- Optimal? No - – returns the first solution it finds

Depth-limited search

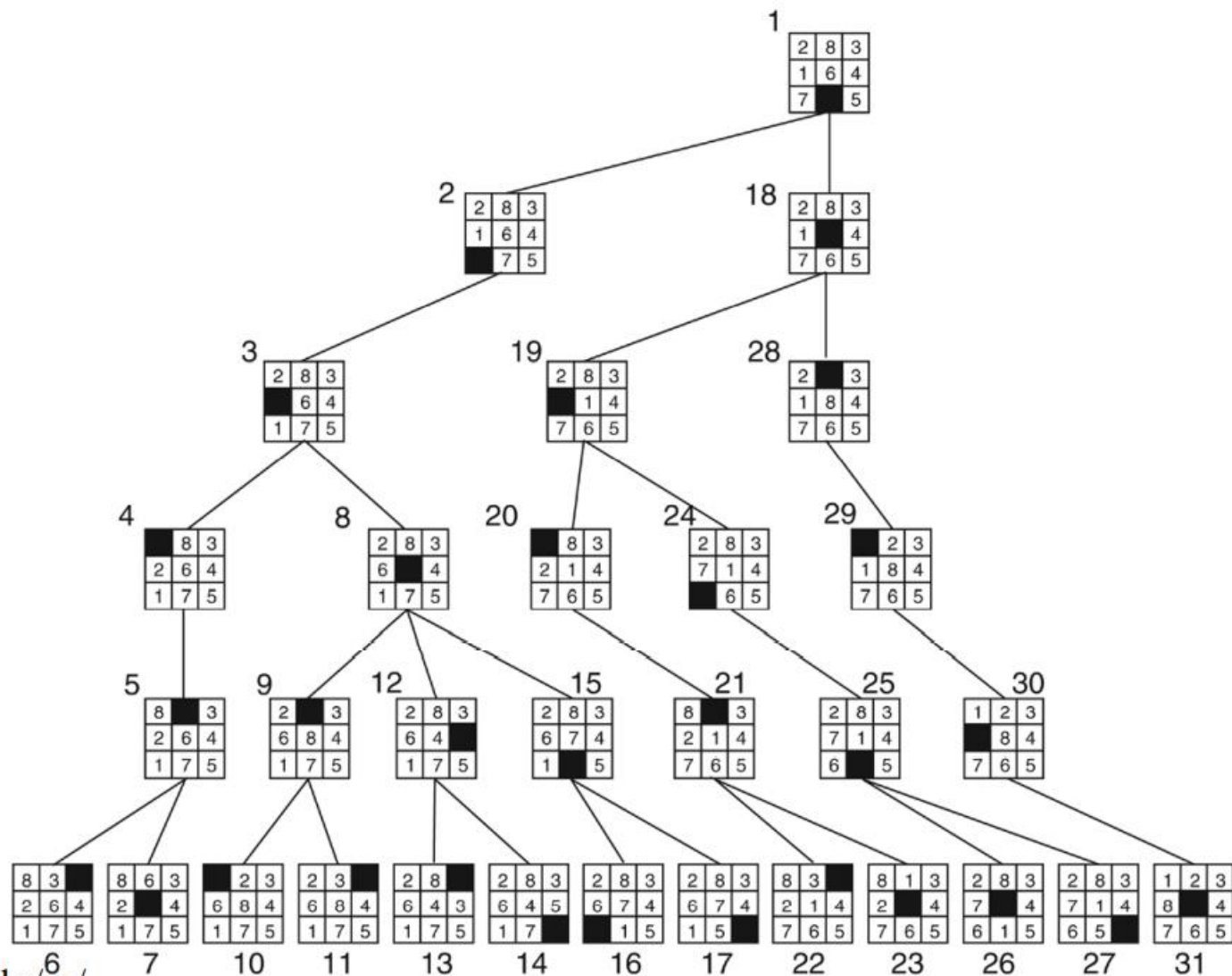
= depth-first search with depth limit l ,
i.e., nodes at depth l have no successors

- Recursive implementation:

```
function DEPTH-LIMITED-SEARCH(problem, limit) returns soln/fail/cutoff
    RECURSIVE-DLS(MAKE-NODE(INITIAL-STATE[problem]), problem, limit)

function RECURSIVE-DLS(node, problem, limit) returns soln/fail/cutoff
    cutoff-occurred?  $\leftarrow$  false
    if GOAL-TEST[problem](STATE[node]) then return SOLUTION(node)
    else if DEPTH[node] = limit then return cutoff
    else for each successor in EXPAND(node, problem) do
        result  $\leftarrow$  RECURSIVE-DLS(successor, problem, limit)
        if result = cutoff then cutoff-occurred?  $\leftarrow$  true
        else if result  $\neq$  failure then return result
    if cutoff-occurred? then return cutoff else return failure
```


Depth limited search



Depth limited search

- Like depth-first search, except:
 - Depth limit in the expand function

L = depth limit

Complete if $L \geq d$ (d is the depth of the shallowest goal)

Not optimal (even if one continues the search after the first solution has been found, because an optimal solution may not be within the depth limit L)

$O(b^L)$ time

$O(bL)$ space

Iterative deepening search

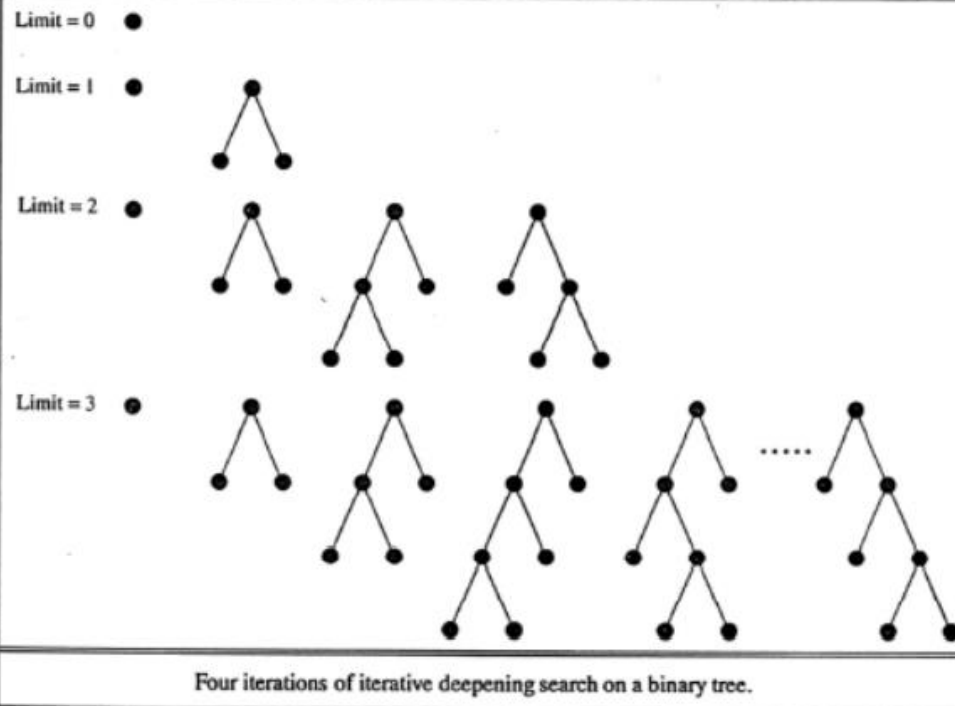
- Use DFS as a subroutine
 1. Check the root
 2. Do a DFS searching for a path of length 1
 3. If there is no path of length 1, do a DFS searching for a path of length 2
 4. If there is no path of length 2, do a DFS searching for a path of length 3...

Iterative deepening search

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution sequence
  inputs: problem, a problem

  for depth  $\leftarrow$  0 to  $\infty$  do
    if DEPTH-LIMITED-SEARCH(problem, depth) succeeds then return its result
  end
  return failure
```

*Depth-first search
for each depth.*



Iterative deepening search

- Iterative deepening search $l=0$, $l=1$, $l=2$

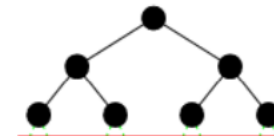
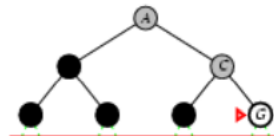
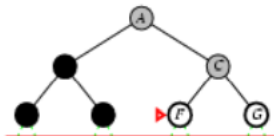
Limit = 0



Limit = 1

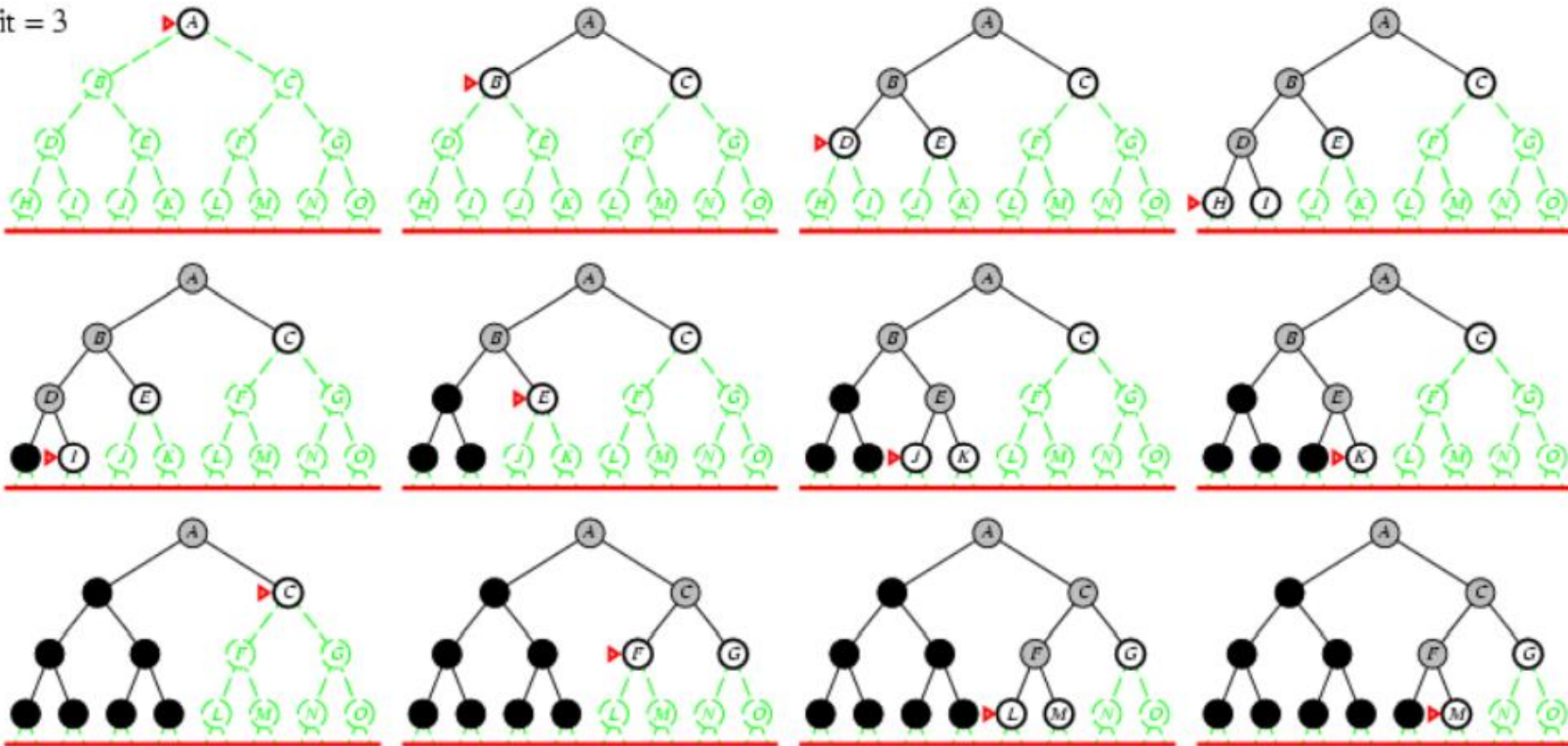


Limit = 2



Iterative deepening search $l=3$

Limit = 3



Iterative deepening search

- Number of nodes generated in a depth-limited search to depth d with branching factor b :

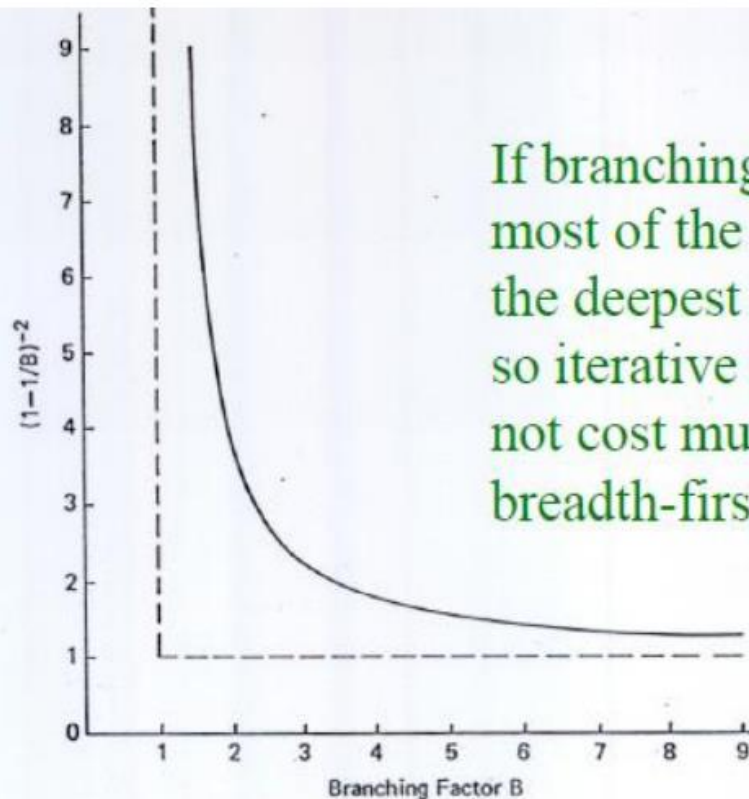
$$N_{DLS} = b^0 + b^1 + b^2 + \dots + b^{d-2} + b^{d-1} + b^d$$

- Number of nodes generated in an iterative deepening search to depth d with branching factor b :

$$N_{IDS} = (d+1)b^0 + d b^1 + (d-1)b^2 + \dots + 3b^{d-2} + 2b^{d-1} + 1b^d$$

- For $b = 10$, $d = 5$,
 - $N_{DLS} = 1 + 10 + 100 + 1,000 + 10,000 + 100,000 = 111,111$
 - $N_{IDS} = 6 + 50 + 400 + 3,000 + 20,000 + 100,000 = 123,456$
- Overhead = $(123,456 - 111,111)/111,111 = 11\%$

Iterative deepening search



If branching factor is *large*, most of the work is done at the deepest level of search, so iterative deepening does not cost much relatively to breadth-first search.

Graph of branching factor vs. constant coefficient as search depth goes to infinity.

Conclusion:

- Iterative deepening is preferred when search space is large and depth of (optimal) solution is unknown
- Not preferred if branching factor is tiny (near 1)

Properties of iterative deepening search

- Complete? Yes
- Time? $(d+1)b^0 + d b^1 + (d-1)b^2 + \dots + b^d = O(b^d)$
- Space? $O(bd)$
- Optimal? Yes, if step cost = 1

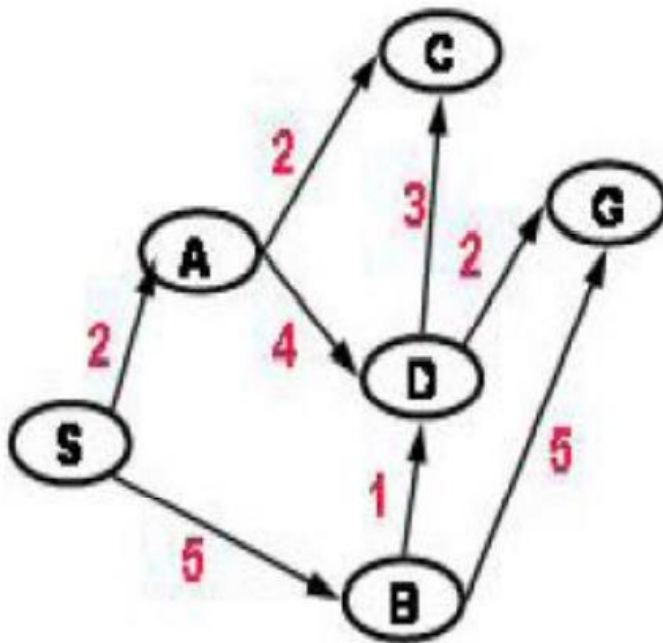
Uniform-cost search

- For each frontier node, save the total cost of the path from the initial state to that node
- Expand the frontier node with the lowest path cost
- Implementation: *frontier* is a priority queue ordered by path cost
- Equivalent to breadth-first if step costs all equal

Uniform-cost search

Each link has a length or cost (which is always greater than 0)

We want shortest or least cost path



Total path cost:

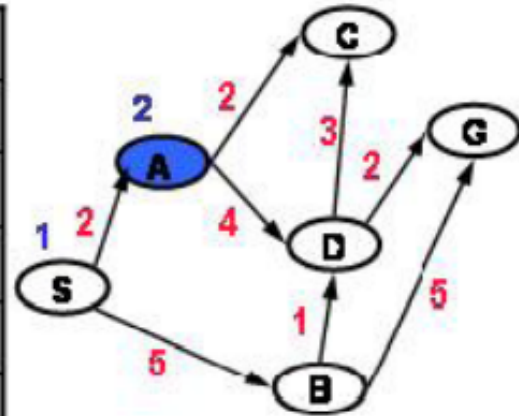
(S A C) 4

(S B D G) 8

(S A D C) 9

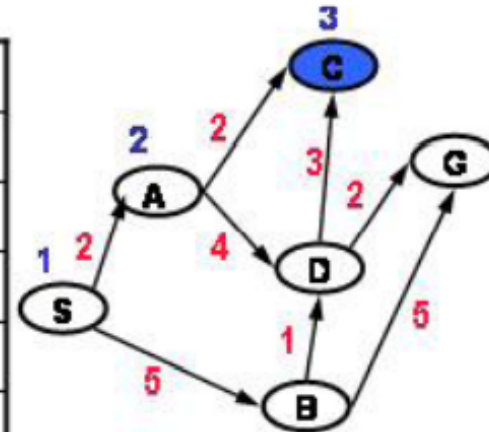
Uniform-cost search

	Q
1	(0 S)
2	(2 A S) (5 B S)



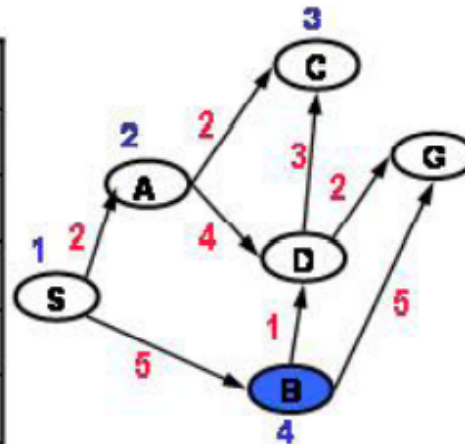
Uniform-cost search

	Q
1	(0 S)
2	(2 A S) (5 B S)
3	(4 C A S) (6 D A S) (5 B S)



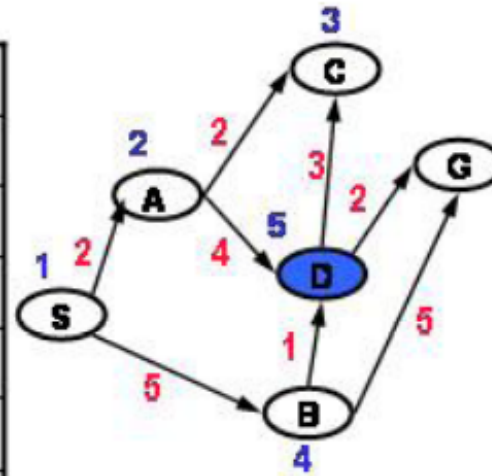
Uniform-cost search

	Q
1	(0 S)
2	(2 A S) (5 B S)
3	(4 C A S) (6 D A S) (5 B S)
4	(6 D A S) (5 B S)



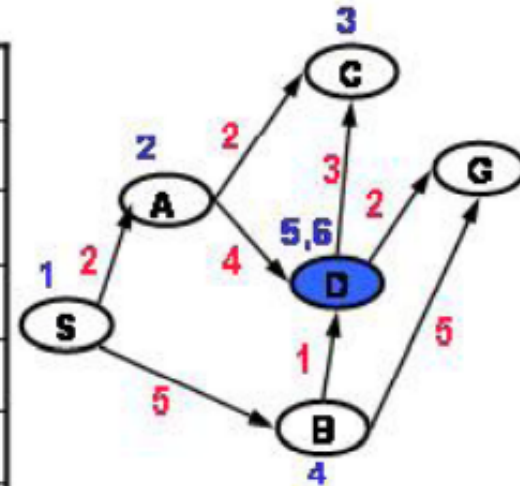
Uniform-cost search

	Q
1	<u>(0 S)</u>
2	<u>(2 A S)</u> (5 B S)
3	<u>(4 C A S)</u> (6 D A S) (5 B S)
4	(6 D A S) <u>(5 B S)</u>
5	<u>(6 D B S)</u> (10 G B S) (6 D A S)



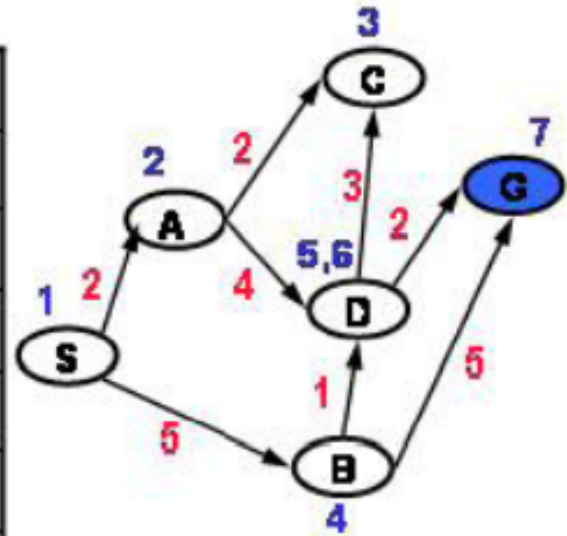
Uniform-cost search

	Q
1	<u>(0 S)</u>
2	<u>(2 A S)</u> (5 B S)
3	<u>(4 C A S)</u> (6 D A S) (5 B S)
4	(6 D A S) <u>(5 B S)</u>
5	<u>(6 D B S)</u> (10 G B S) (6 D A S)
6	(8 G D B S) (9 C D B S) (10 G B S) <u>(6 D A S)</u>



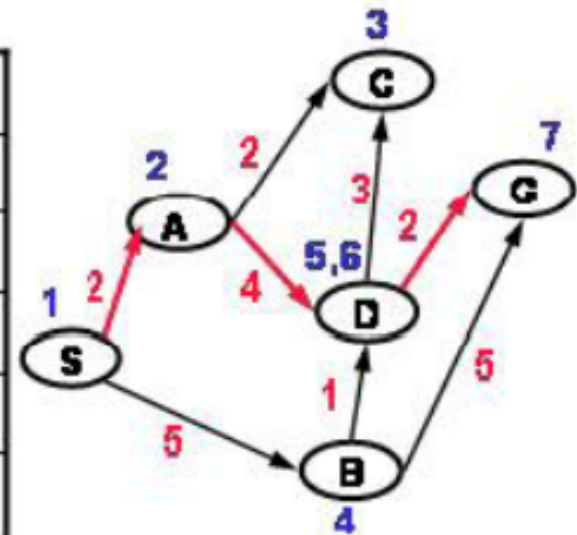
Uniform-cost search

	Q
1	<u>(0 S)</u>
2	<u>(2 A S)</u> (5 B S)
3	<u>(4 C A S)</u> (6 D A S) (5 B S)
4	(6 D A S) <u>(5 B S)</u>
5	<u>(6 D B S)</u> (10 G B S) (6 D A S)
6	(8 G D B S) (9 C D B S) (10 G B S) <u>(6 D A S)</u>
7	(8 G D A S) (9 C D A S) (8 G D B S) (9 C D B S) (10 G B S)



Uniform cost search

	Q
1	<u>(0 S)</u>
2	<u>(2 A S)</u> (5 B S)
3	<u>(4 C A S)</u> (6 D A S) (5 B S)
4	(6 D A S) <u>(5 B S)</u>
5	<u>(6 D B S)</u> (10 G B S) (6 D A S)
6	(8 G D B S) (9 C D B S) (10 G B S) <u>(6 D A S)</u>
7	<u>(8 G D A S)</u> (9 C D A S) (8 G D B S) (9 C D B S) (10 G B S)

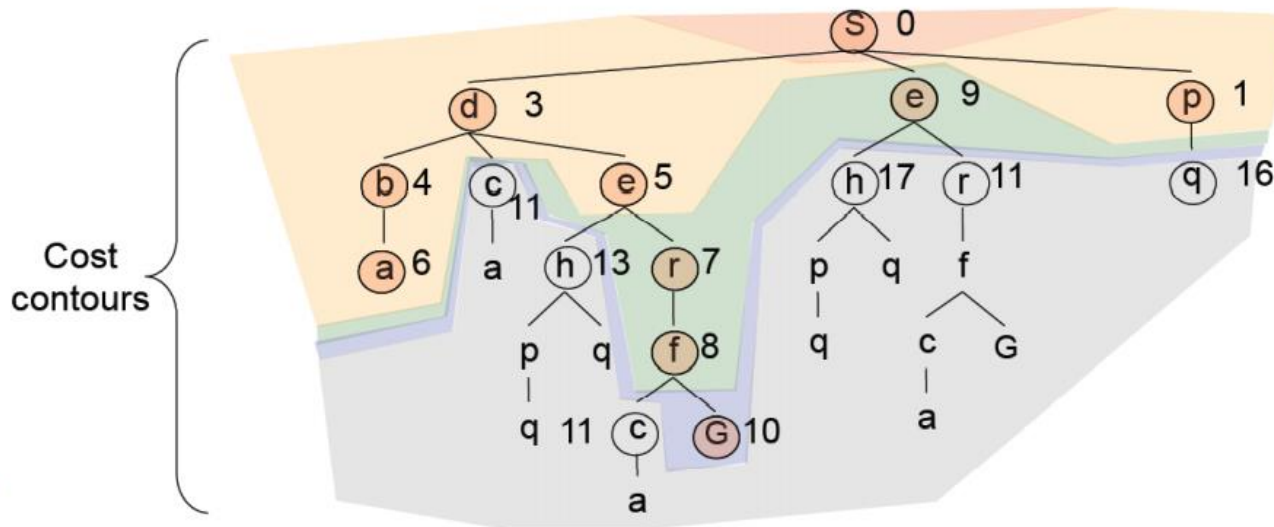
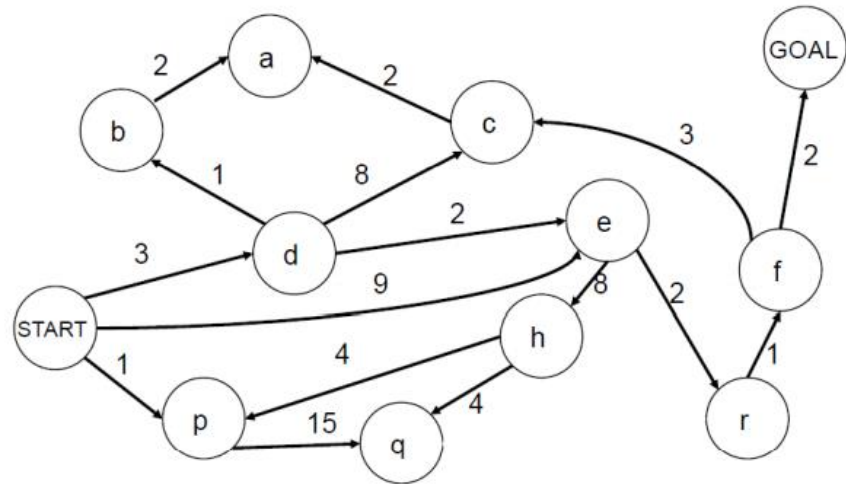


Why not stop on the first goal

- When doing Uniform Cost, it is not correct to stop the search when the first path to a goal is generated, that is, when a node whose state is a goal is added to Q.
- We must wait until such a path is pulled off the Q and tested in step 3. It is only at this point that we are sure it is the shortest path to a goal since there are no other shorter paths that remain unexpanded.
- This contrasts with the Any Path searches where the choice of where to test for a goal was a matter of convenience and efficiency, not correctness.
- In the previous example, a path to G was generated at step 5, but it was a different, shorter, path at step 7 that we accepted.

Uniform-cost search example

- Expansion order:
(S,p,d,b,e,a,r,f,e,G)



Properties of uniform-cost search

- Expand least-cost unexpanded node
- **Implementation:**
 - *fringe* = queue ordered by path cost
- Equivalent to breadth-first if step costs all equal
- **Complete?**

Yes, if step cost is greater than some positive constant ϵ (we don't want infinite sequences of steps that have a finite total cost)
- **Optimal?**

Yes

Uniform-cost search

Complete? Yes, if step cost $\geq \epsilon$

Time? # of nodes with *pathn cost* $g \leq$ cost of optimal solution,

$O(b^{\text{ceiling}(C^*/\epsilon)})$ where C^* is the cost of the optimal solution

This can be greater than $O(b^d)$: the search can explore long paths consisting of small steps before exploring shorter paths consisting of larger steps

Space? # of nodes with $g \leq$ cost of optimal solution,
 $O(b^{\text{ceiling}(C^*/\epsilon)})$

Optimal? Yes – nodes expanded in increasing order of $g(n)$

Review: Uninformed search strategies

Algorithm	Complete?	Optimal?	Time complexity	Space complexity
BFS	Yes	If all step costs are equal	$O(b^d)$	$O(b^d)$
DFS	No	No	$O(b^m)$	$O(bm)$
IDS	Yes	If all step costs are equal	$O(b^d)$	$O(bd)$
UCS	Yes	Yes	Number of nodes with $g(n) \leq C^*$	

b: maximum branching factor of the search tree

d: depth of the optimal solution

m: maximum length of any path in the state space

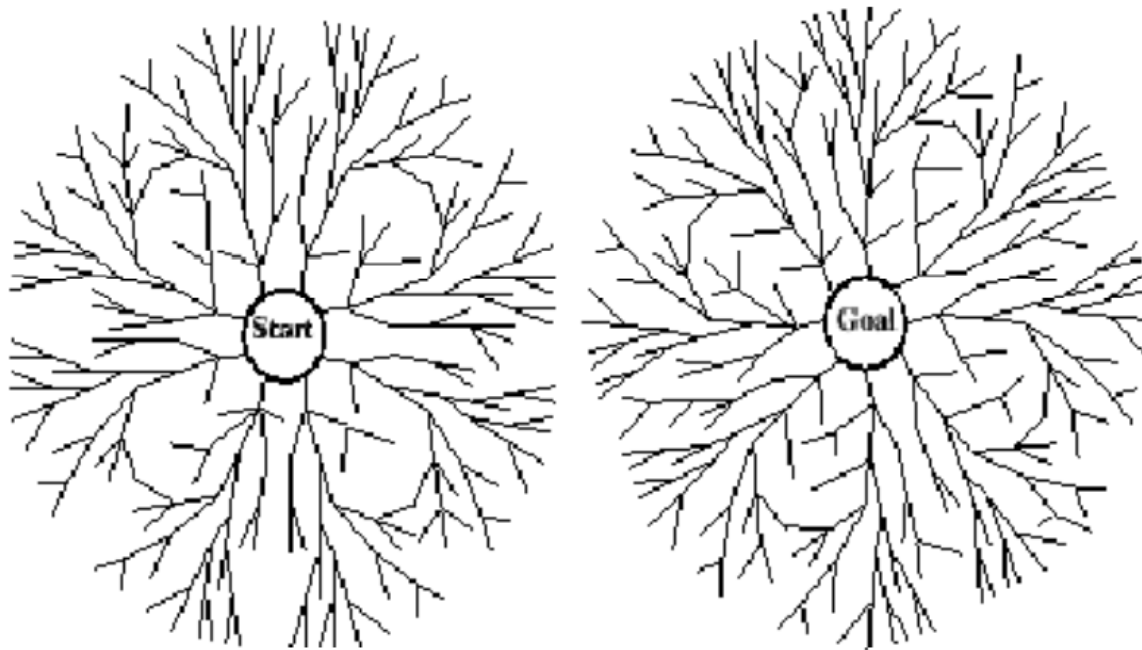
C^* : cost of optimal solution

$g(n)$: cost of path from start state to node n

Bidirectional search

- Run two simultaneous searches – one forward from the initial state, and the other backward from the goal, stopping when two searches meet in the middle

$$b^{d/2} + b^{d/2} < b^d$$



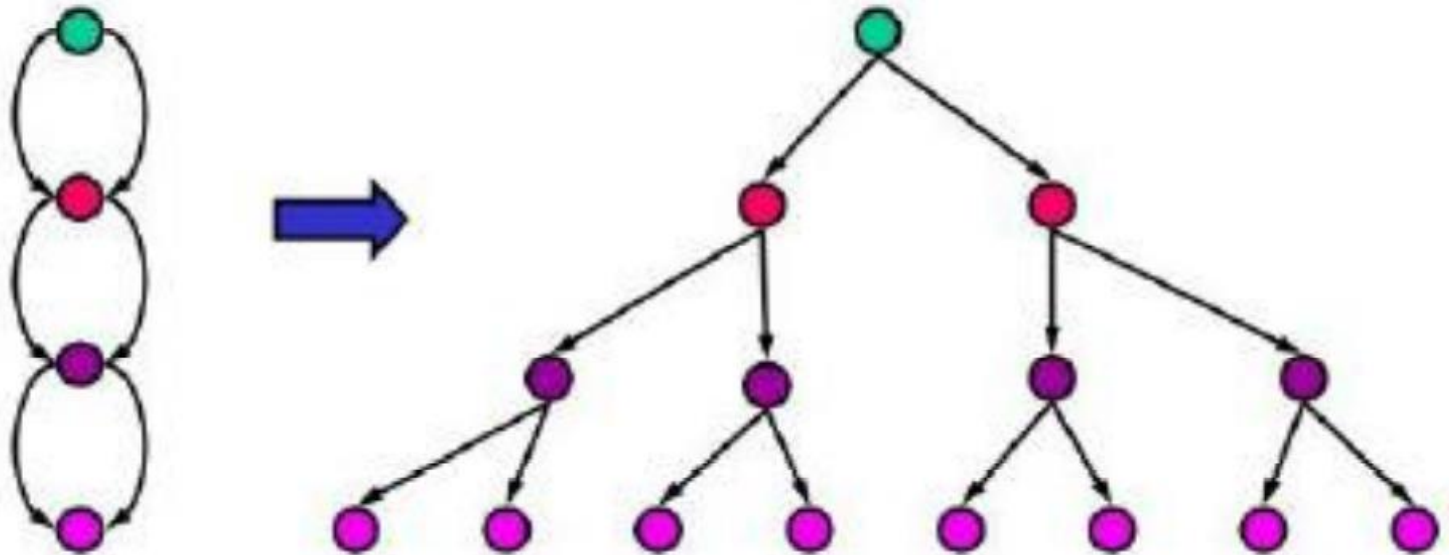
For $b = 10$, $d = 6$, BFS 1,111, 111 nodes, BS 2,222 nodes

Bi-directional Search discussion

- Need to have operators that calculate **predecessors**.
- Need a way to check that **states meet (are the same)**
- **Efficient way to check when searches meet: hash table**
- **Can use IDS or BFS or DFS in each half**
- Optimal, complete, $O(b^{d/2})$ time.
- Still $O(b^{d/2})$ space (even with iterative deepening)
because the nodes of at least one of the searches have to
be stored to check matches.

Repeated states

- Failure to detect repeated states can turn a linear problem into an exponential one!



Handling repeated states

Do not return to the state just you came from

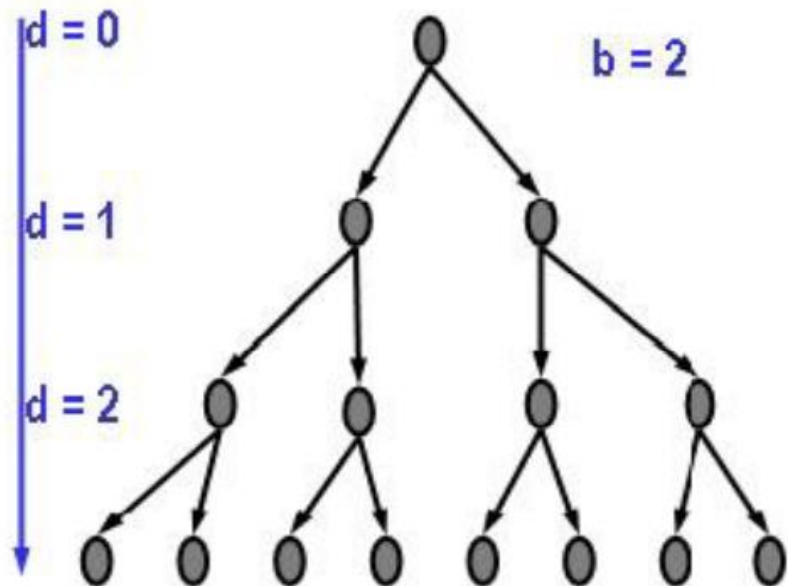
Do not create paths with cycles in them

Do not generate any state that was ever generated before

Worst Case Running Time

Max Time $\propto$ Max #Visited

- The number of states in the search space may be exponential in some “depth” parameter, e.g. number of actions in a plan, number of moves in a game.
- All the searches, with or without visited list, may have to visit each state at least once, in the worst case.
- So, all searches will have worst case running times that are at least proportional to the total number of states and therefore exponential in the “depth” parameter.



d is depth

b is branching factor

$$b^d < (b^{d+1} - 1) / (b - 1) < b^{d+1}$$

states in tree

Summary

Problem formulation usually requires abstracting away real-world details to define a state space that can feasibly be explored

Variety of uninformed search strategies

Iterative deepening search uses only linear space and not much more time than other uninformed algorithms

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Time	b^d	b^d	b^m	b^l	b^d	$b^{d/2}$
Space	b^d	b^d	bm	bl	bd	$b^{d/2}$
Optimal?	Yes	Yes	No	No	Yes	Yes
Complete?	Yes	Yes	No	Yes, if $l \geq d$	Yes	Yes

b = branching factor

d = depth of shallowest goal state

m = maximum depth of the search tree

l = depth limit of the algorithm

Informed search strategies

Informed search strategies use problem specific knowledge beyond the definition of the problem itself

Idea: give the algorithm “hints” about the desirability of different states

- Use an *evaluation function* to rank nodes and select the most promising one for expansion

Greedy best-first search

A* search

Informed search strategies

Idea: use an **evaluation function** $f(n)$ to select the node for expansion

- estimate of "desirability"
- Expand most desirable unexpanded node

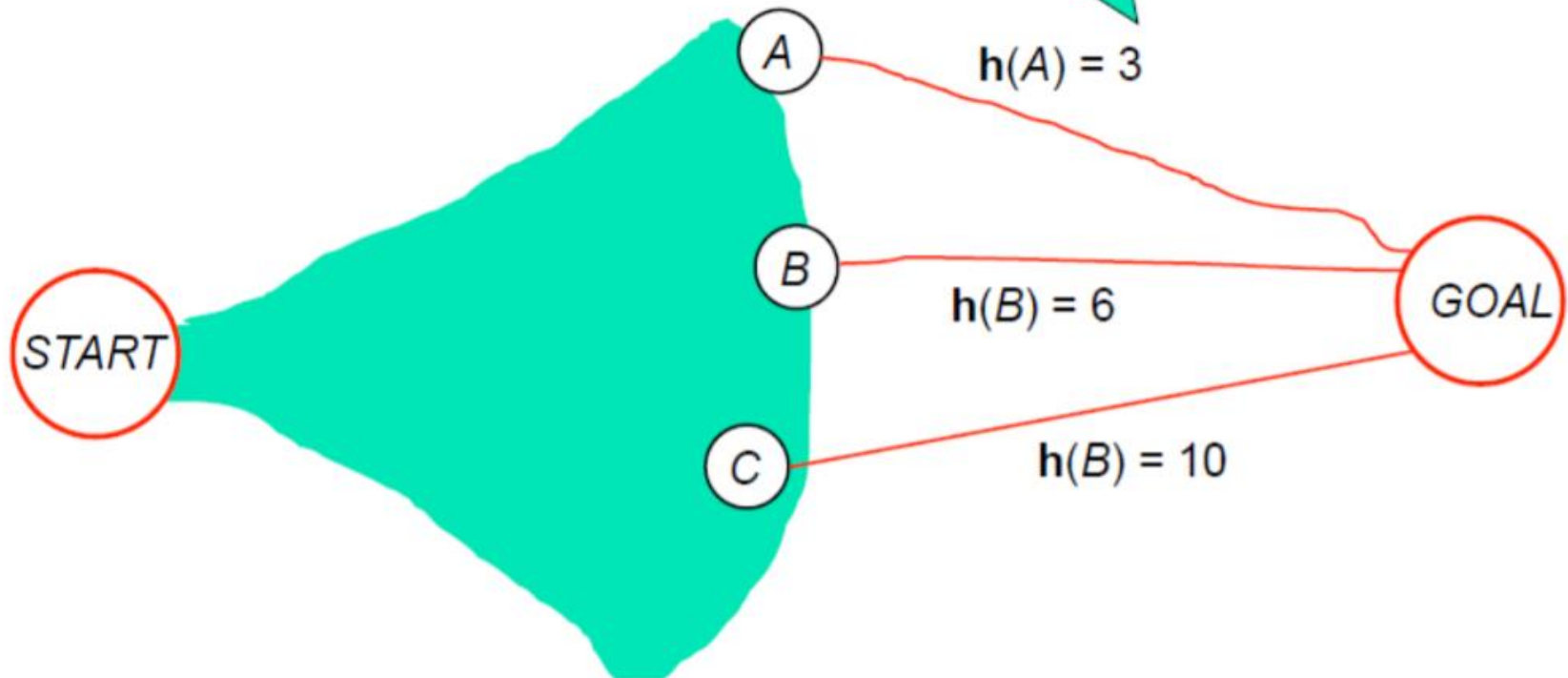
Implementation:

Order the nodes in fringe in decreasing order of desirability

Informed – Estimate cost to the goal

- Introduce a function **$h(s)$** to estimate the unknown distance from state s to the goal

GUESS how close a node is to the GOAL - *A is closest*

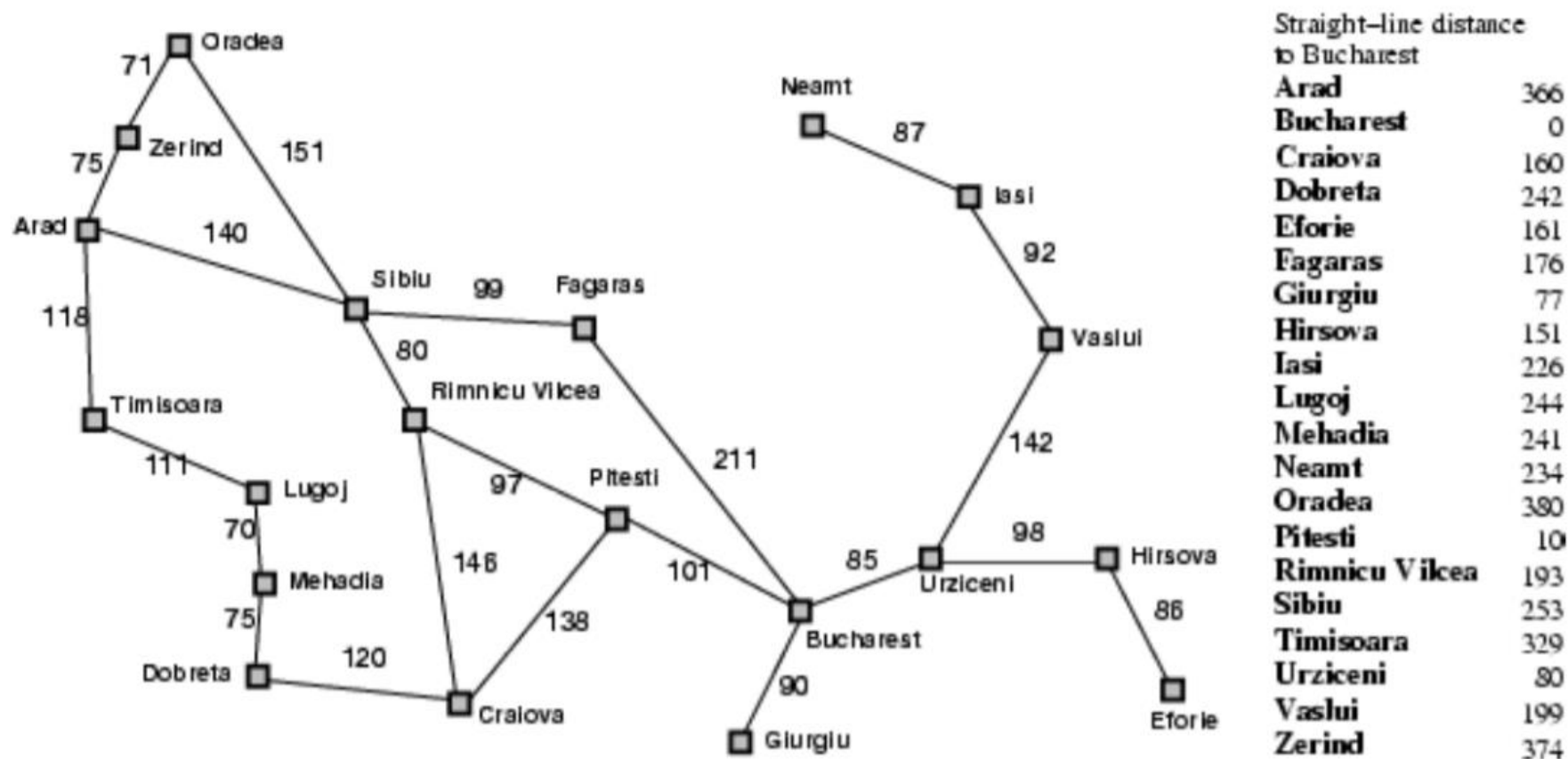


Greedy best-first search

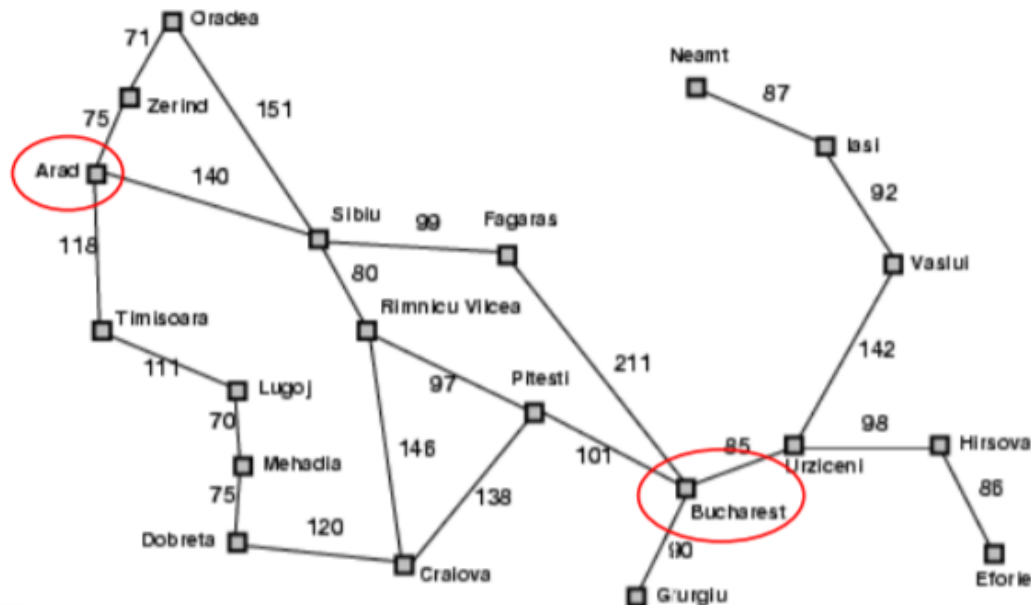
- Greedy best-first search expands the node that **appears** to be closest to goal
 - Evaluation function $f(n) = h(n)$ (heuristic)
 - = estimate of cost from n to *goal*
 - e.g., $h_{SLD}(n)$ = straight-line distance from n to Bucharest
 - Note that, h_{SLD} cannot be computed from the problem description itself. It takes a certain amount of experience to know that it is correlated with actual road distances, and therefore it is a useful heuristic
-

Romania with step costs in km

e.g. For Romania, cost of the cheapest path from Arad to Bucharest can be estimated via the straight line distance

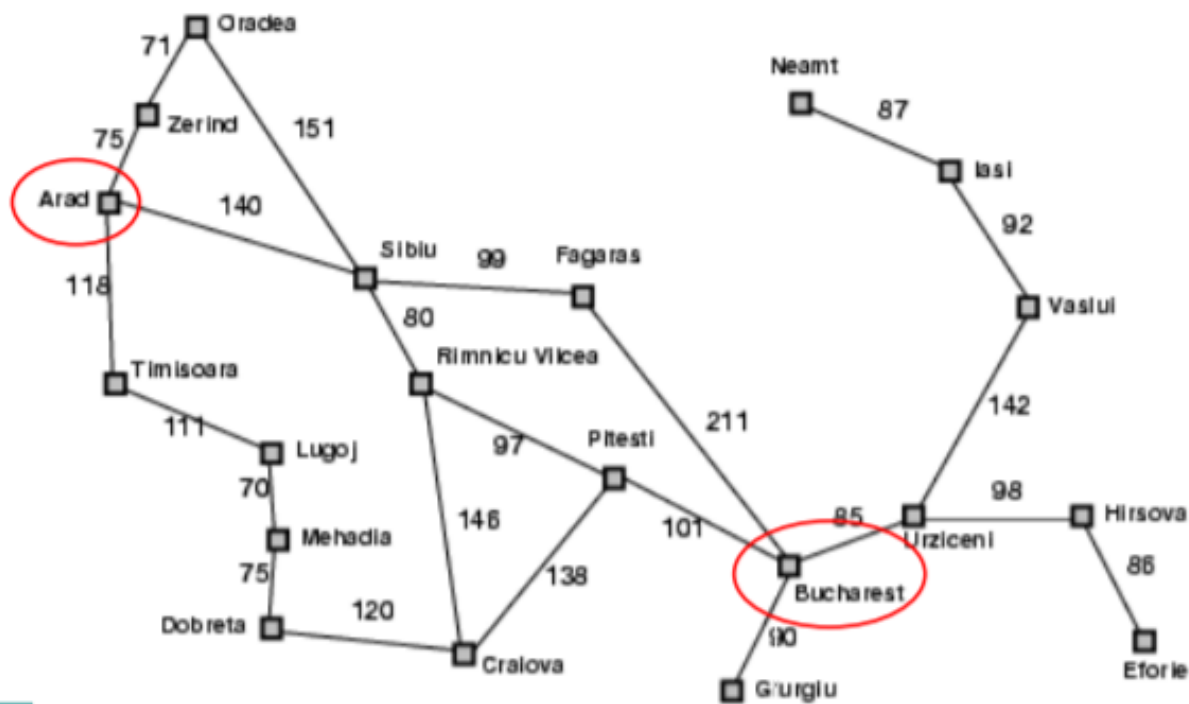


Greedy best -first search example



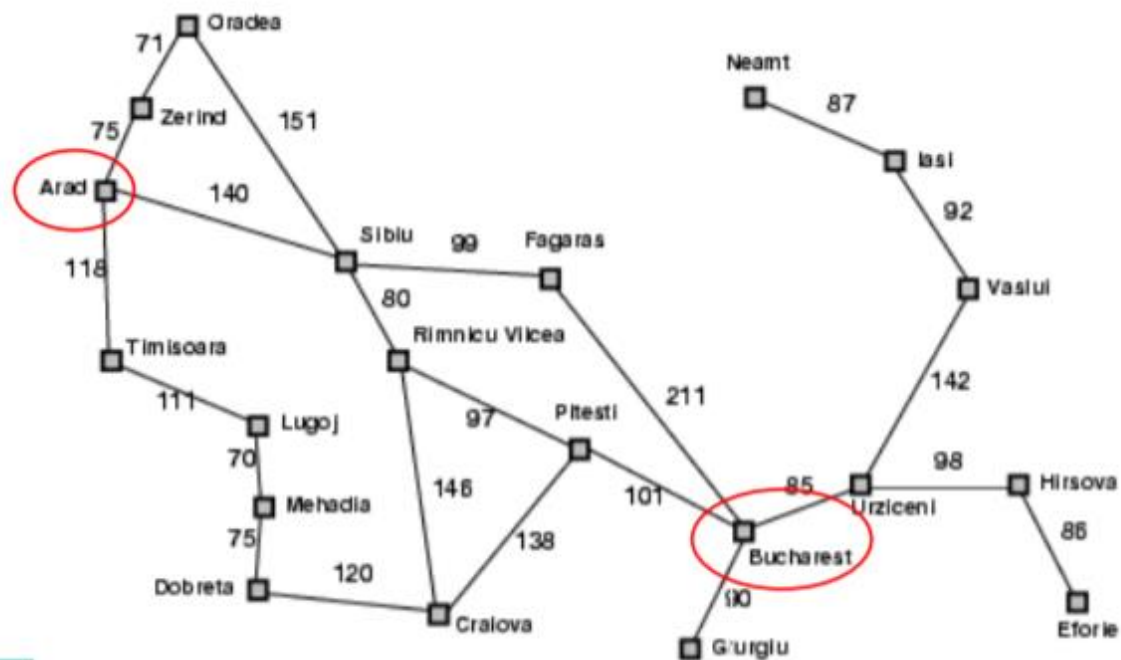
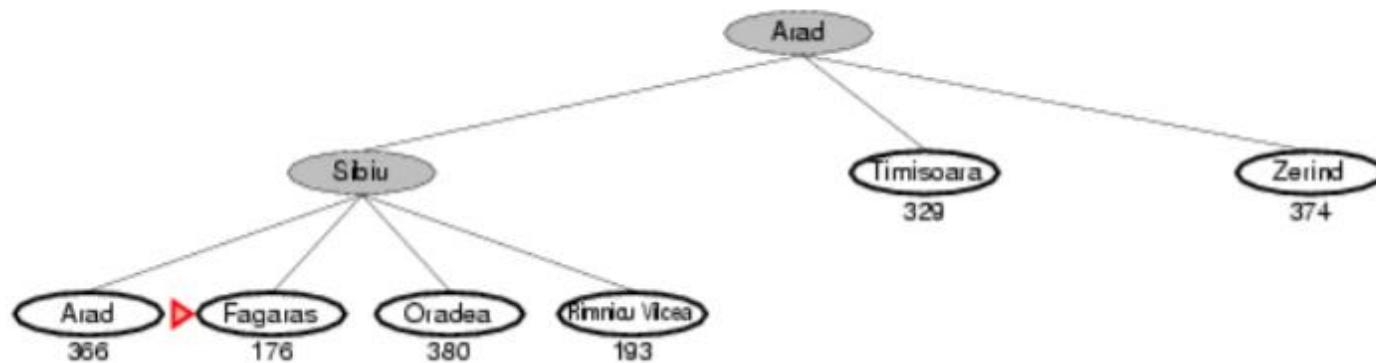
Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



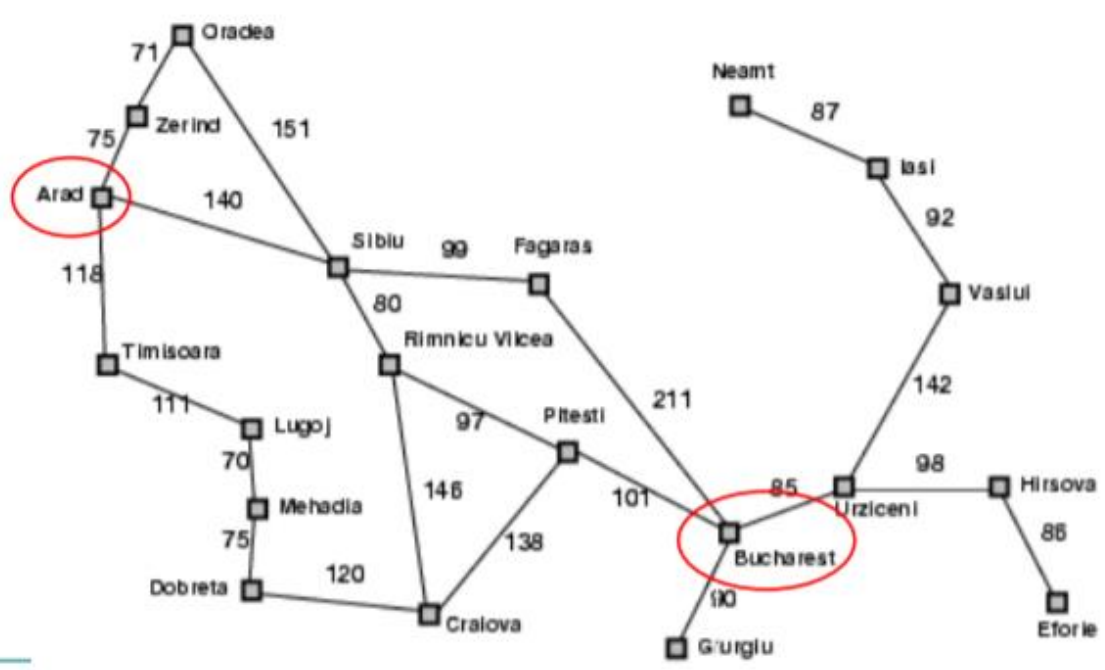
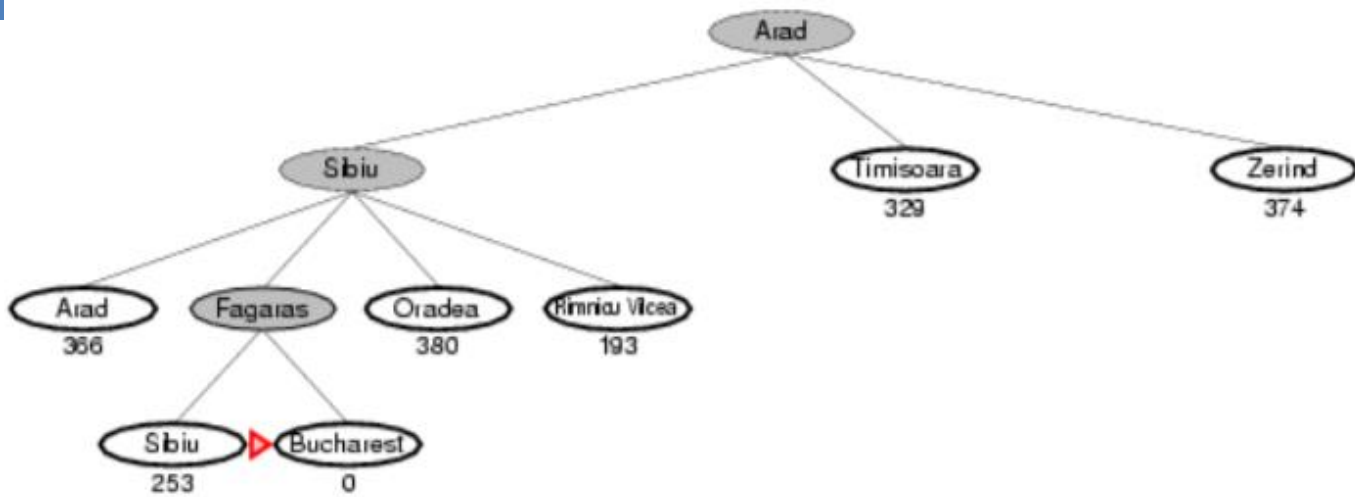
Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



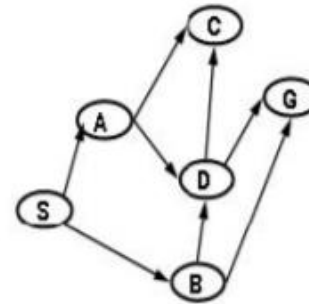
Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Greedy best-first search – Another example

Pick “best” (by heuristic value) element of Q; Add path extensions anywhere in Q

	Q	Visited
1	(10 S)	S
2		
3		
4		
5		



Heuristic Values

A=2 C=1 S=10

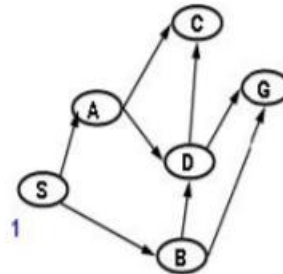
B=3 D=4 G=0

Added paths in **blue**; heuristic value of node's state is in front.

We show the paths in **reversed** order; the node's state is the first entry.

Greedy best-first search – Another example

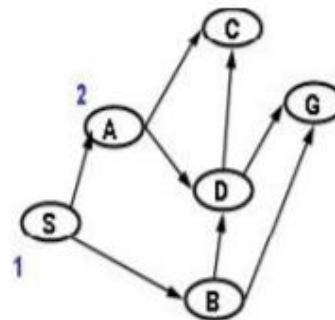
	Q	Visited
1	(10 S)	S
2	(2 A S) (3 B S)	A,B,S
3		
4		
5		



Heuristic Values

A=2 C=1 S=10
B=3 D=4 G=0

	Q	Visited
1	(10 S)	S
2	(2 A S) (3 B S)	A,B,S
3	(1 C A S) (3 B S) (4 D A S)	C,D,B,A,S
4		
5		

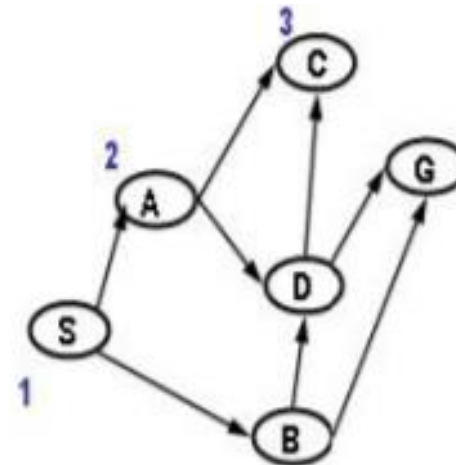


Heuristic Values

A=2 C=1 S=10
B=3 D=4 G=0

Greedy best-first search – Another example

	Q	Visited
1	(10 S)	S
2	(2 A S) (3 B S)	A,B,S
3	(1 C A S) (3 B S) (4 D A S)	C,D,B,A,S
4	(3 B S) (4 D A S)	C,D,B,A,S
5		

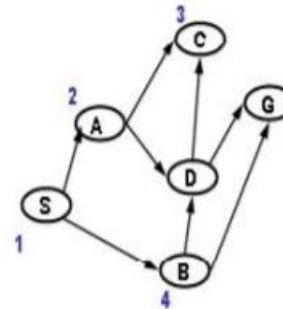


Heuristic Values

A=2 C=1 S=10
B=3 D=4 G=0

Greedy best-first search – Another example

	Q	Visited
1	(10 S)	S
2	(2 A S) (3 B S)	A,B,S
3	(1 C A S) (3 B S) (4 D A S)	C,D,B,A,S
4	(3 B S) (4 D A S)	C,D,B,A,S
5	(0 G B S) (4 D A S)	G,C,D,B,A,S

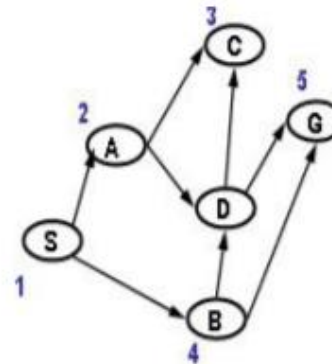


Heuristic Values

A=2 C=1 S=10

B=3 D=4 G=0

	Q	Visited
1	(10 S)	S
2	(2 A S) (3 B S)	A,B,S
3	(1 C A S) (3 B S) (4 D A S)	C,D,B,A,S
4	(3 B S) (4 D A S)	C,D,B,A,S
5	(0 G B S) (4 D A S)	G,C,D,B,A,S



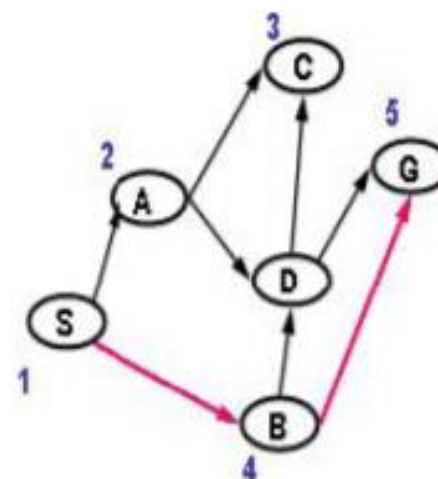
Heuristic Values

A=2 C=1 S=10

B=3 D=4 G=0

Greedy best-first search – Another example

	Q	Visited
1	(10 S)	S
2	(2 A S) (3 B S)	A,B,S
3	(1 C A S) (3 B S) (4 D A S)	C,D,B,A,S
4	(3 B S) (4 D A S)	C,D,B,A,S
5	(0 G B S) 4 D A S)	G,C,D,B,A,S



Heuristic Values

A=2 C=1 S=10

B=3 D=4 G=0

Greedy best first search video

- <https://www.youtube.com/watch?v=dv1m3L6QXWs>

Properties of greedy best-first search

- **Complete?**
No – can get stuck in loops

Path through Faragas is not the optimal

In getting Iasi to Faragas, it will expand Neamt first but it is a dead end



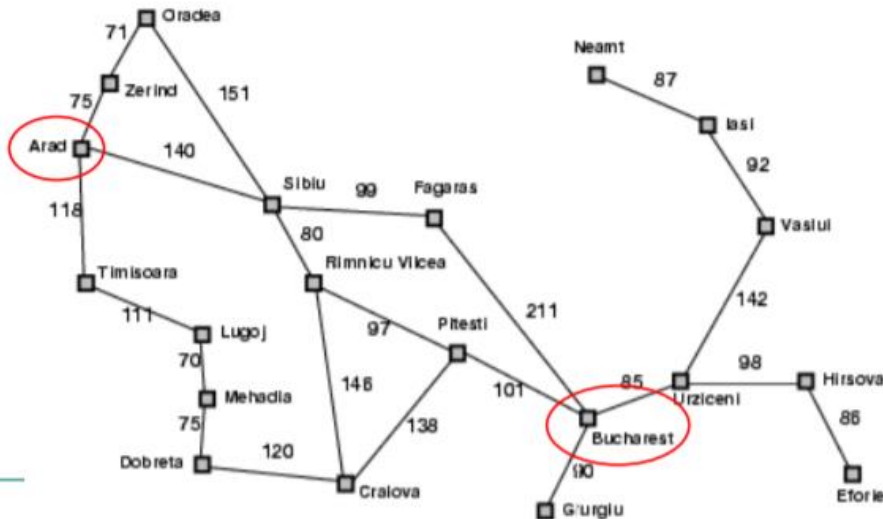
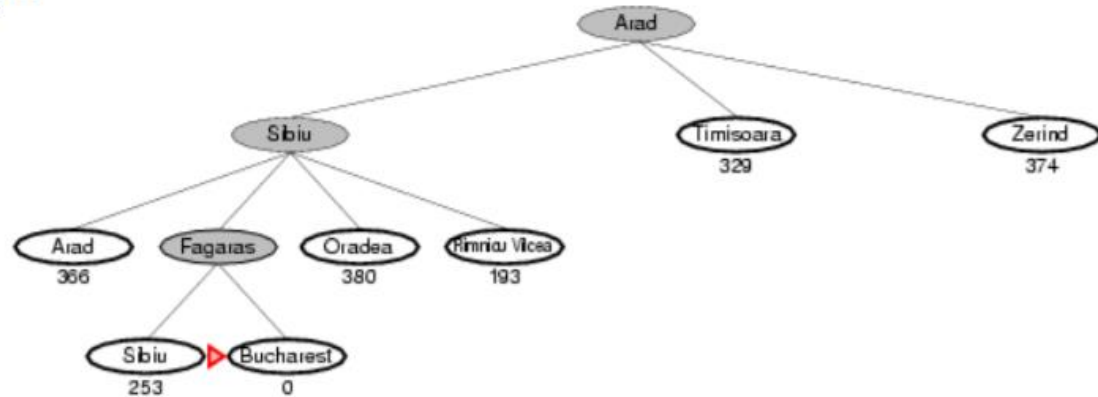
Properties of greedy best-first search

- Complete?**

No – can get stuck in loops

- Optimal?**

No



Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Properties of greedy best-first search

Complete?

No – can get stuck in loops

Optimal?

No

Time?

Worst case: $O(b^m)$

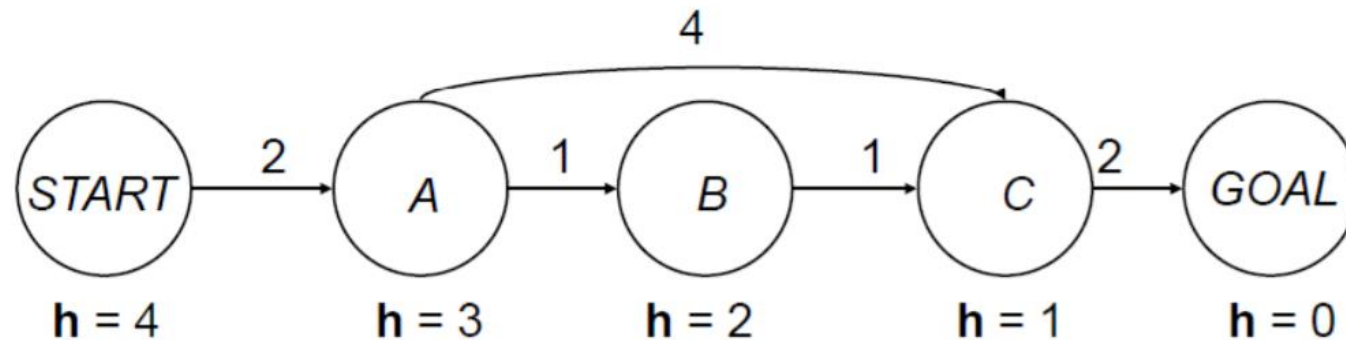
Can be much better with a good heuristic

Space?

Worst case: $O(b^m)$

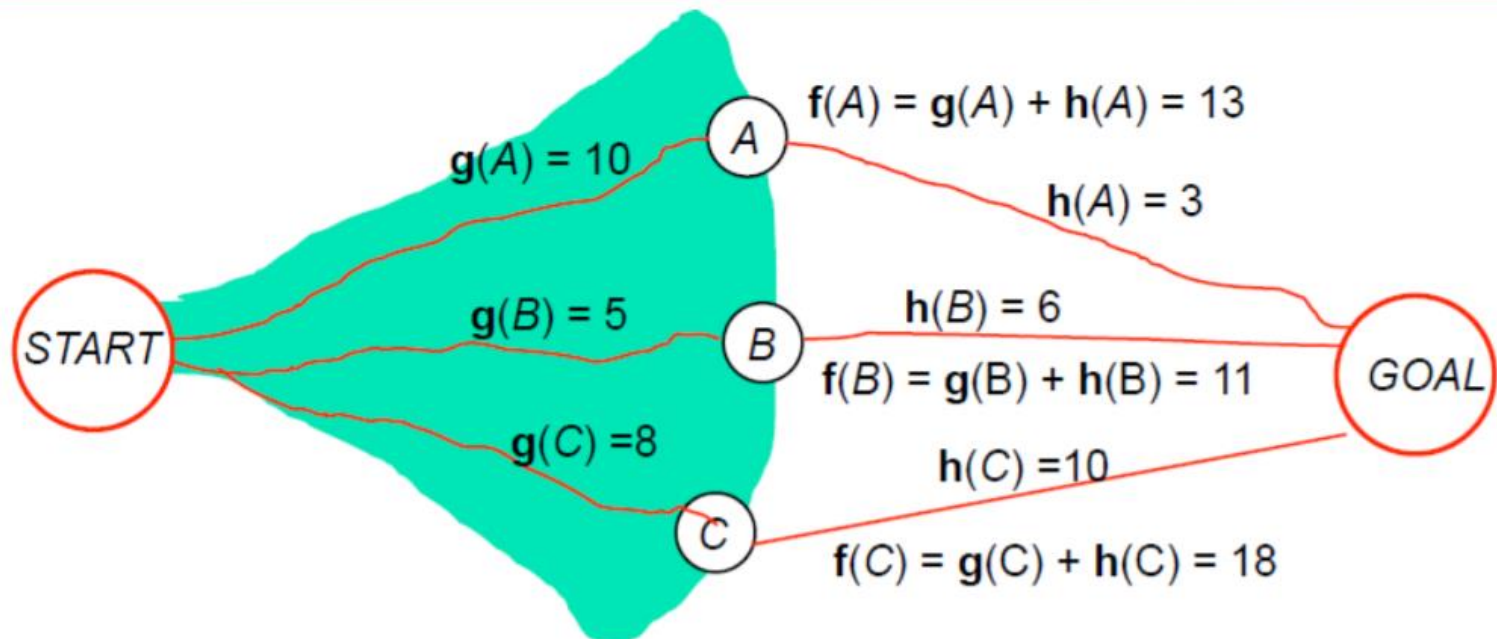
keeps all nodes in memory

How can we fix the greedy problem?



- What solution do we find in this case?
 - (START,4)
 - (A,3)
 - (C,1), (B,2)
 - (Goal,0) START-A-C-Goal
 - Greedy search clearly not optimal, even though the heuristic function is “good.”
 - How about keeping track of the distance already traveled in addition to the distance remaining?
-

Fixing the problem



- **$g(s)$** is the (shortest cost so far) from *START* to *s* only
- **$h(s)$ estimates** the cost from *s* to *GOAL*
- Key insight: **$g(s) + h(s)$** estimates the **total** cost of the cheapest path from *START* to *GOAL* going through *s*
- → **A*** algorithm

A* search

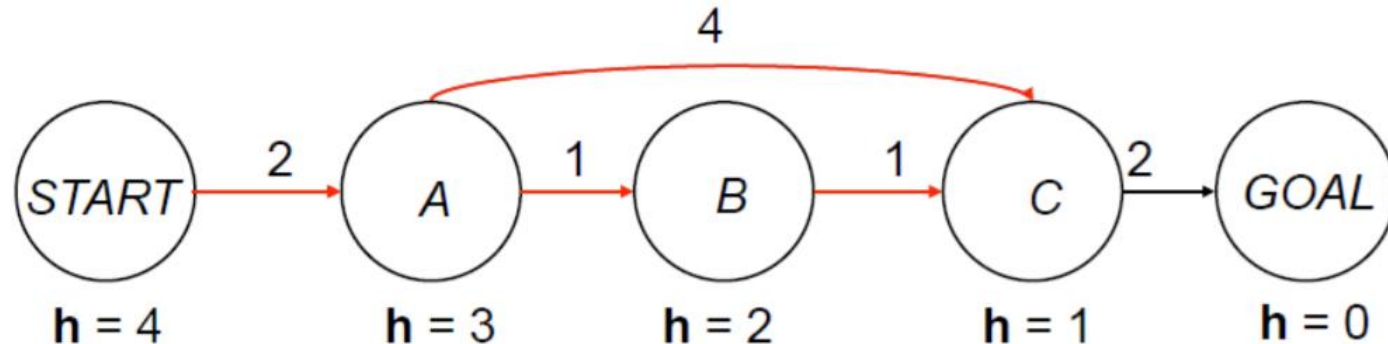
- Idea: avoid expanding paths that are already expensive
- The **evaluation function** $f(n)$ is the estimated total cost of the path through node n to the goal:

$$f(n) = g(n) + h(n)$$

$g(n)$ = cost so far to reach n

$h(n)$ = estimated cost from n to goal

Can A* fix the problem?



$\{(START, 4)\}$

$\{(A, 5)\}$

$$(f(A) = g(A) + h(A) = g(START) + \text{cost}(START, A) + 3 = 0 + 2 + 3)$$

$\{(B, 5) (C, 7)\}$

$$(f(C) = g(C) + h(C) = g(A) + \text{cost}(A, C) + 1 = 2 + 4 + 1)$$

$\{(C, 5)\}$

$$(f(C) = g(C) + h(C) = g(B) + \text{cost}(B, C) + 1 = 3 + 1 + 1)$$

$\{(GOAL, 6)\}$

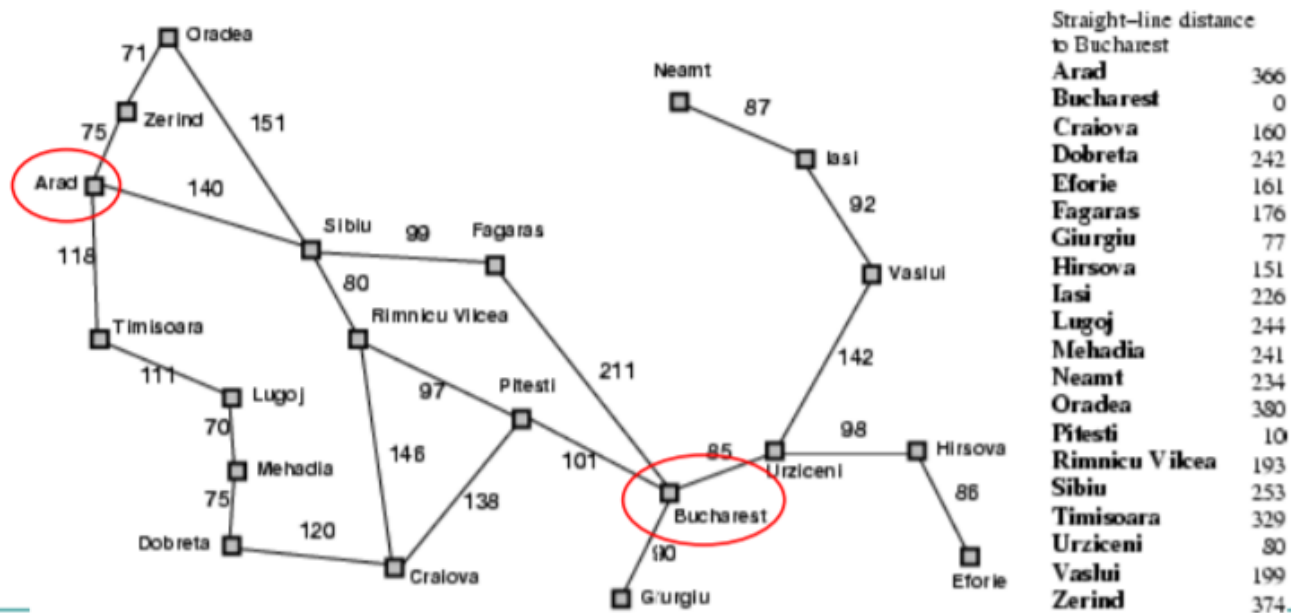
A* search example

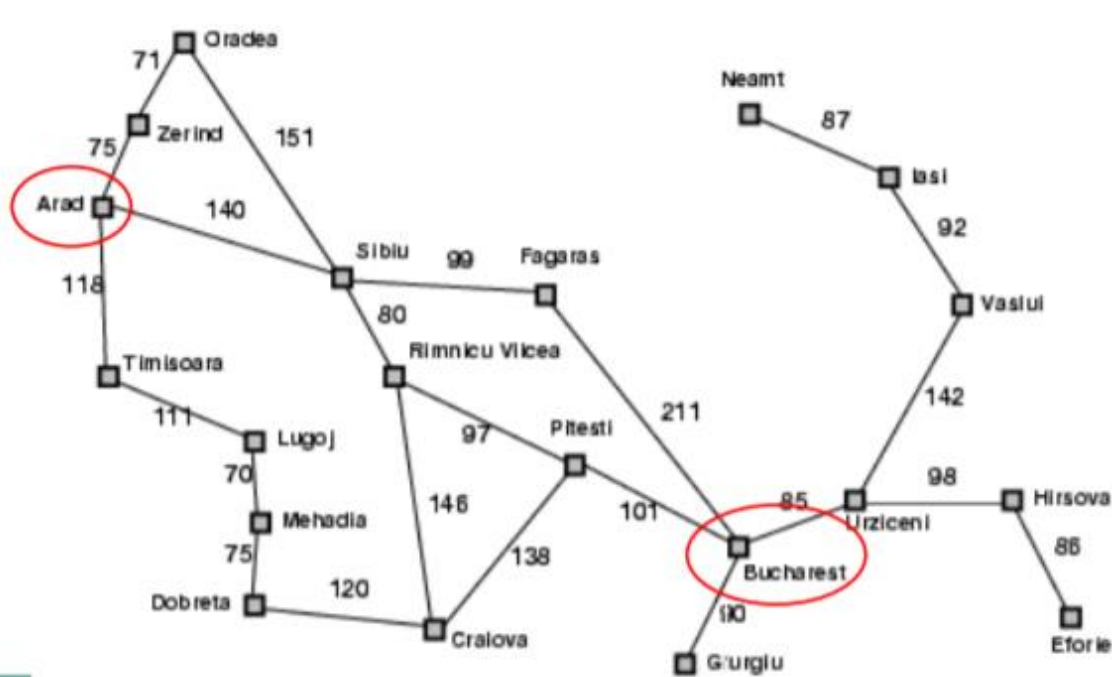
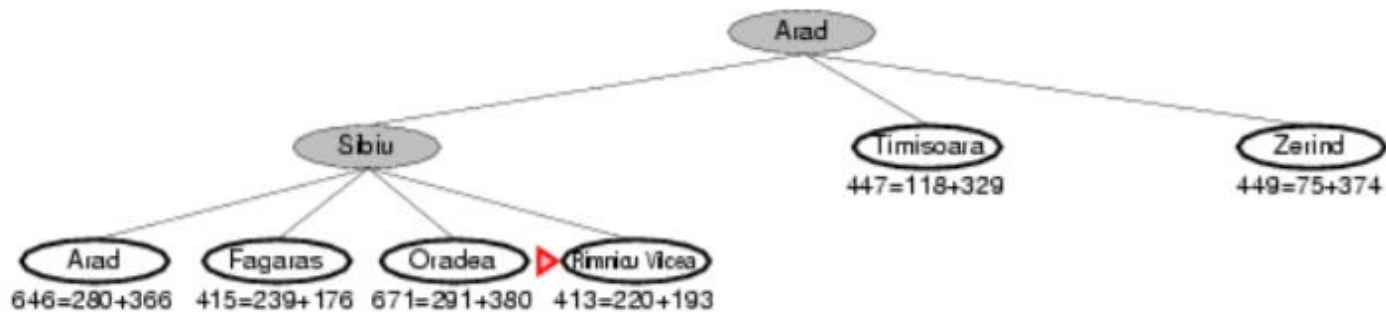


Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

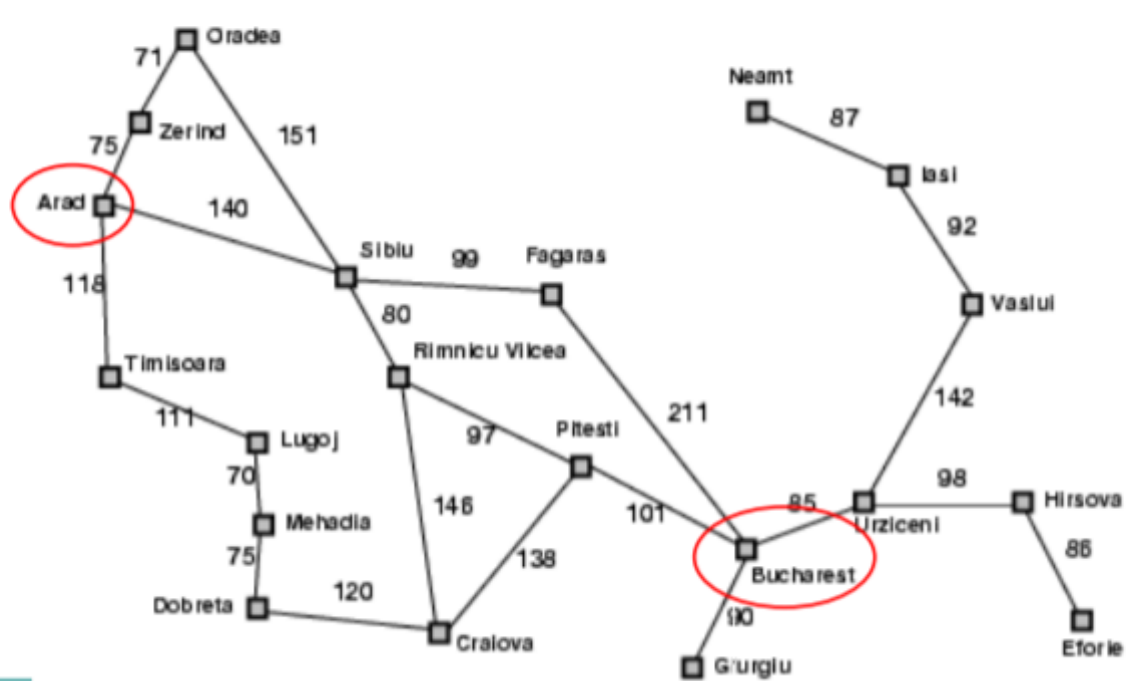
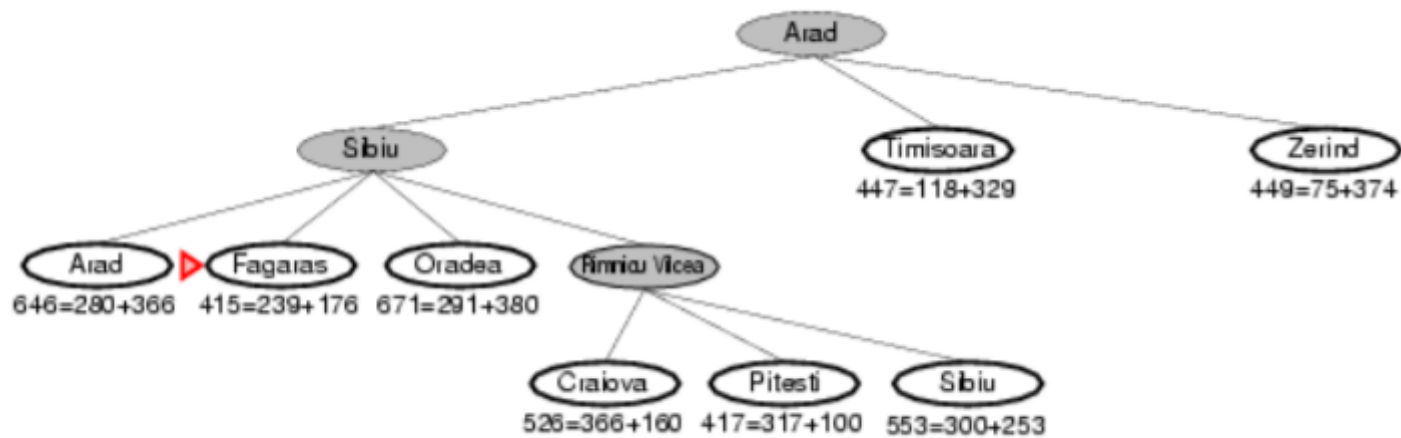
A* search example





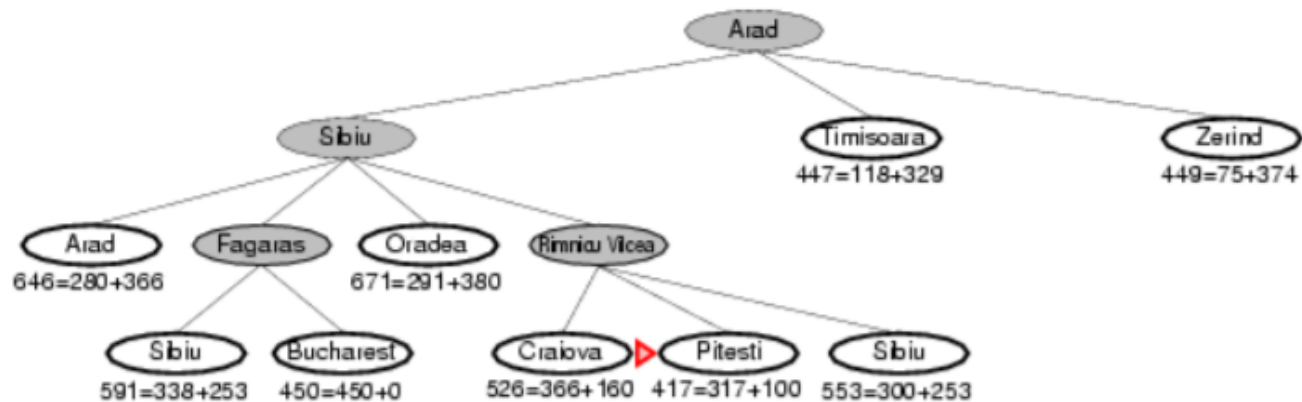
Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



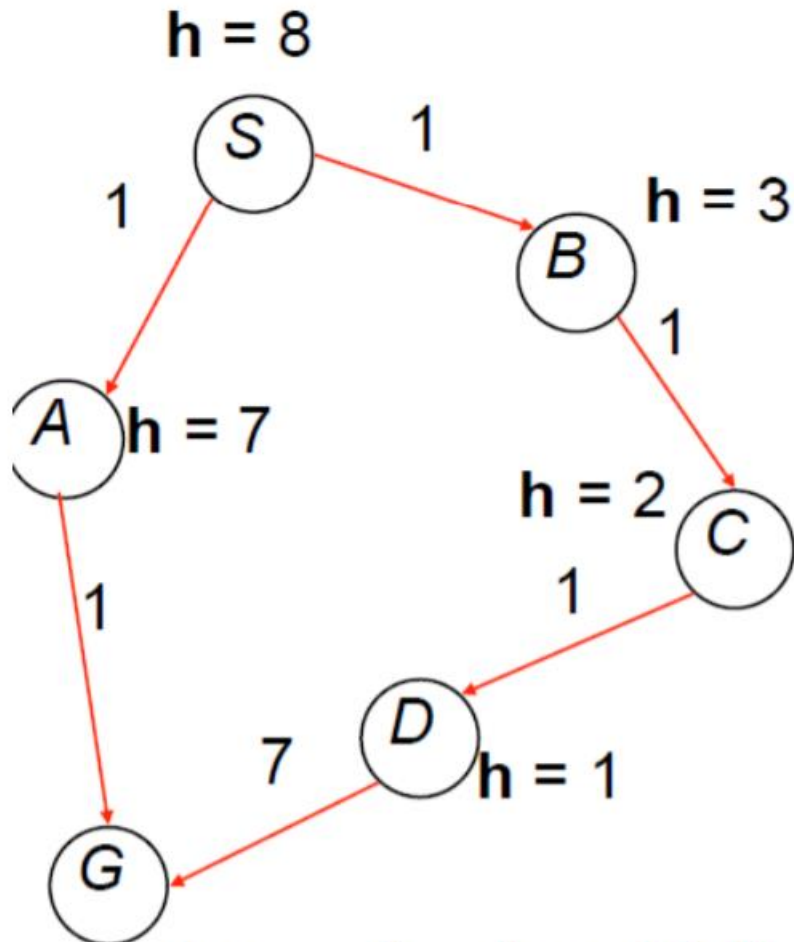
Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

When terminate?



Queue:

$\{(B,4), (A,8)\}$

$\{(C,4), (A,8)\}$

$\{(D,4), (A,8)\}$

$\{(A,8), (G,10)\}$

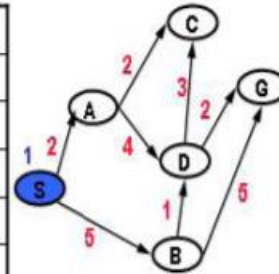
We have encountered G.
Should we stop?

- Stop only when GOAL is **popped (lowest f)** from the queue

A* search – Another example

Pick best (by path length+heuristic) element of Q; Add path extensions anywhere in Q

	Q
1	<u>(0 S)</u>



Heuristic Values

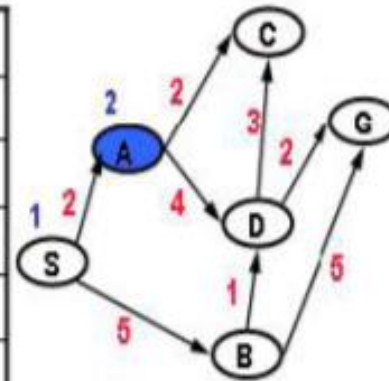
A=2 C=1 S=0
B=3 D=1 G=0

Added paths in **blue**; underlined paths are chosen for extension.

We show the paths in **reversed** order; the node's state is the first entry.

A* search – Another example

	Q
1	(0 S)
2	(4 A S) (8 B S)



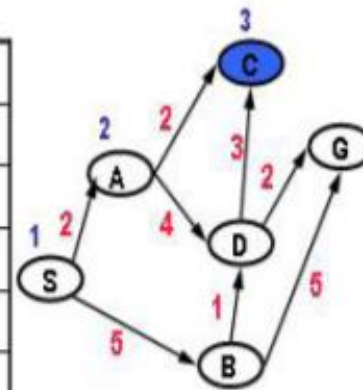
Heuristic Values

A=2 C=1 S=0

B=3 D=1 G=0

A* search – Another example

	Q
1	(0 S)
2	(4 A S) (8 B S)
3	(5 C A S) (7 D A S) (8 B S)



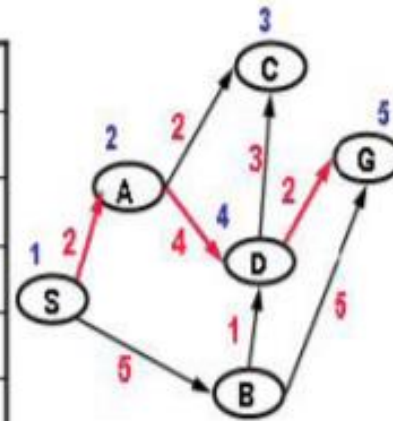
Heuristic Values

A=2 C=1 S=0

B=3 D=1 G=0

A* search – Another example

	Q
1	(0 S)
2	(4 A S) (8 B S)
3	(5 C A S) (7 D A S) (8 B S)
4	(7 D A S) (8 B S)
5	(8 G D A S) (10 C D A S) (8 B S)



Heuristic Values

A=2 C=1 S=0
B=3 D=1 G=0

Admissible Heuristic

- Let $h^*(N)$ be the cost of the optimal path from N to a goal node
- The heuristic function $h(N)$ is **admissible** if:

$$0 \leq h(N) \leq h^*(N)$$

- An admissible heuristic function is always **optimistic** !

G is a goal node $\rightarrow h(G) = 0$